

Operating System — ISRO CSE Revision Notes

Concise notes + relevant exam questions, compressed from the KnowledgeGate Operating System module.

1. Introduction to OS — Types, Structure & System Calls

1.1 What is an OS?

- **OS** = system software acting as an intermediary between user and hardware; a **resource manager/allocator** (CPU time, memory, I/O devices, buses) and an **extended machine** (hides hardware complexity, provides a convenient platform for applications).
- **Goals:** Primary = **Convenience** (user-friendly). Secondary = **Efficiency, Reliability, Maintainability**. Functions are the means; goals are the ends.

1.2 Evolution & Types of OS

Type	Core Idea	Key Trade-off
Batch OS	Similar jobs grouped and run as a batch with no user interaction; a Batch Monitor sequences them	Improves turnaround vs manual operation, but no interactivity; CPU still idles during I/O
Spooling	(Simultaneous Peripheral Operations On-Line) Data queued in a buffer/disk for a slow device (e.g., printer); overlaps I/O of one job with CPU of another	Lets CPU keep working instead of waiting on slow I/O devices
Multiprogramming	Several jobs held in main memory at once; when the running job blocks (e.g., on I/O), CPU switches to another ready job	Keeps CPU busy as long as ANY job is ready — but needs more complex memory/scheduling management
Multitasking / Time-sharing	Multiprogramming + very fast context switching so each user feels they have the whole machine to themselves	Enables interactivity; overhead = frequent context switches
Multiprocessing (tightly coupled)	Two or more CPUs share one main memory	Higher throughput/reliability, but added complexity and contention overhead
Real-Time OS (RTOS)	Deterministic, deadline-driven scheduling	Hard RT vs Soft RT (see below)
Distributed OS	Independent, networked, loosely-coupled nodes (no shared memory/clock) presented to the user as ONE single system (transparency / single-system-image)	Enables resource sharing, speedup, reliability, communication — at the cost of network dependency

- **TRAP: Multitasking and Time-sharing are essentially the same concept** (both = multiprogramming + fast switching for interactive use) — do not treat them as unrelated OS categories.
- **Multiprocessing types:**
- **Symmetric (SMP):** all CPUs are **peers**, each runs an identical copy of the OS, no boss-worker relation. Most modern systems (Windows/Linux/macOS).
- **Asymmetric: master-slave** — one master CPU schedules/controls; others (slaves) only execute assigned work. Simpler to design but less efficient (master can bottleneck).
- **Multiprocessing advantages:** increased throughput (speed-up with N processors is **less than N** due to coordination overhead — **TRAP:** don't assume linear speedup), economy of scale (shared peripherals/storage), increased reliability/fault tolerance (losing 1 of 10 CPUs slows the system ~10%, doesn't crash it), less power/heat per unit of work.
- **RTOS:** Hard RT = missing a deadline is a **total failure** (airbags, anti-lock brakes — reacts at t=0). Soft RT = missing a deadline **degrades quality** but is tolerable (digital cameras, streaming — reacts at t=0+).

1.3 User Interfaces

Interface	Style	Typical User
CLI (Command-Line)	Text commands via a shell/interpreter	Sysadmins, power users — efficient once learned
GUI	Mouse, windows, icons, menus	General users — intuitive, visually guided
Touchscreen	Gesture-based	Mobile/tablet users

1.4 OS Structure

Approach	Idea	Trade-off
Simple/Monolithic	No well-defined structure; everything runs in kernel space (e.g., original MS-DOS)	Fast (no IPC overhead) but hard to maintain/extend, poor fault isolation
Layered	OS broken into layers 0 (hardware) → N (user interface); each layer only uses services of the layer directly below it	Modular, easier to debug/verify (information hiding); but added layer-crossing overhead
Microkernel	Kernel keeps only bare essentials (e.g., IPC, basic scheduling); most services (file system, drivers) run as user-space servers	Smaller kernel → more reliable & easier to extend (new services need no kernel changes) but slower due to frequent user-kernel IPC (e.g., Mach, MINIX)

- **TRAP:** Microkernel = **more** reliable/extensible but **slower**; Monolithic = **faster** but less modular/reliable — students often flip this trade-off.

1.5 System Calls & Dual Mode

- **System call:** the interface through which a user program requests an OS service (implemented via an **API** — e.g., POSIX API for UNIX/Linux/macOS, Windows API, Java API). The caller only needs to know the API contract, not the internal implementation.
- **System call categories:** Process control, **File management**, **Device management**, **Information maintenance** (get/set time, system data, process/file attributes), **Communication** (create/delete connections, send/receive messages).
- **Dual Mode & Mode Bit:** hardware includes a **mode bit** — **0 = Kernel/Supervisor mode**, **1 = User mode**. Purpose: protect the OS and hardware from buggy or malicious user programs by restricting which instructions user code may execute directly (e.g., I/O instructions are privileged).
- **Mechanism:** user program executes a system call → **trap (software interrupt)** switches CPU from user mode → kernel mode → OS services the request → control returns to user mode.
- **TRAP:** A **normal function call** does NOT change the mode bit; only a **system call (via trap/interrupt)** switches user mode ↔ kernel mode.

Questions

Q1. Which of the following best defines an Operating System's primary goal?

- A. Maximum efficiency at all costs B. Convenience / user-friendliness, with efficiency as a secondary goal C. Minimum memory footprint D. Maximum throughput only

Answer: B. **Why:** Convenience (usability) is the primary goal; efficiency, reliability, and maintainability are secondary goals that support it.

Q2. [Trap] "Multitasking" and "Time-sharing" as OS concepts are:

- A. Completely unrelated B. Essentially the same concept — multiprogramming plus fast context switching for interactive use C. Time-sharing is older and unrelated to multiprogramming D. Multitasking only applies to single-CPU systems and time-sharing only to multi-CPU systems

Answer: B. **Why:** Both describe multiple users/programs sharing the CPU via frequent switching, giving each the illusion of dedicated access.

Q3. In Symmetric Multiprocessing (SMP), the relationship between processors is:

- A. Boss-worker (master-slave) B. Peers — no processor controls another, each runs an identical OS copy C. Only one processor may ever execute kernel code D. Processors do not communicate

Answer: B. **Why:** SMP treats all CPUs as equals; Asymmetric multiprocessing is the one with a master-slave relationship.

Q4. A pacemaker's control software, where missing a deadline can cause catastrophic failure, requires:

- A. Soft Real-Time OS B. Hard Real-Time OS C. Batch OS D. Time-sharing OS

Answer: B. **Why:** Hard RT systems treat any missed deadline as total failure — appropriate for life-critical embedded control.

Q5. [Numerical/Conceptual] With N processors cooperating on a task, the actual speed-up is:

- A. Always exactly N B. Always greater than N C. Less than N, due to coordination overhead and resource contention D. Always exactly 1

Answer: C. **Why:** Overhead from synchronization and shared-resource contention means real speed-up with N processors is sub-linear (less than N).

Q6. Which OS structuring approach moves most services out of the kernel into user-space servers, shrinking the kernel itself?

- A. Monolithic B. Layered C. Microkernel D. Simple structure

Answer: C. **Why:** The microkernel approach keeps only essential functions in the kernel, improving reliability/extensibility at some IPC performance cost.

Q7. [Trap] What actually happens when a user program executes a system call?

- A. Nothing changes; it behaves like a normal function call B. A trap (software interrupt) switches the CPU from user mode to kernel mode to service the request, then returns to user mode C. The mode bit is permanently set to kernel mode D. The program is terminated

Answer: B. **Why:** A system call triggers a trap that temporarily elevates execution to kernel mode for the OS to perform the privileged operation, then control returns to user mode — unlike an ordinary function call, which never changes the mode bit.

Q8. Which system call category includes "get/set system time" and "get process/file attributes"?

- A. Process control B. Device management C. Information maintenance D. Communication

Answer: C. **Why:** Information maintenance calls handle querying/setting system data, time, and process/file/device attributes.

2. Process Management — PCB, States & Scheduling Basics

2.1 Process vs Program

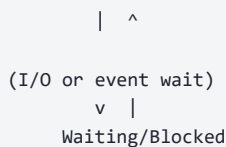
- **Program** = passive entity — an executable file of instructions sitting on disk.
- **Process** = active entity — created when a program is loaded into main memory **and its PCB is created**; requires resources (CPU time, memory, registers).
- Two processes running the same program (e.g., 2 browser windows) are still **entirely separate processes** — same text (code) section, but independent data/heap/stack.
- **Process memory layout (4 sections):** **Text** (code) → **Data** (global variables) → **Heap** (dynamically allocated at runtime) → **Stack** (function parameters, return addresses, local variables).

2.2 Process Control Block (PCB)

- The PCB ("task control block") is the OS's **repository of all information specific to one process**. Standard contents:
 - Process state (new/ready/running/waiting/terminated)
 - Program counter (address of next instruction to execute)
 - CPU registers (accumulators, index registers, stack pointers, condition codes)
 - CPU-scheduling information (priority, scheduling queue pointers)
 - Memory-management information (base/limit registers, page/segment tables)
 - Accounting information (CPU used, time limits, process ID)
 - I/O status information (list of open files, allocated I/O devices)
- **TRAP:** The PCB is exactly what gets saved/restored during a **context switch** — "what triggers/comprises a context switch" questions map directly to this list (program counter + CPU registers being the most time-critical part to save).

2.3 Process States & Lifecycle

New → Ready → Running → Terminated



- **New:** being created. **Ready:** in memory, waiting for CPU allocation. **Running:** instructions executing. **Waiting/Blocked:** waiting on an event (I/O completion, signal). **Terminated:** finished execution, PCB/resources deallocated.
- Transition triggers: Ready→Running = **dispatch** (scheduler picks it). Running→Ready = **timeout/interrupt** (time slice expires). Running→Waiting = **I/O or event wait requested**. Waiting→Ready = **I/O or event completion**. Running→Terminated = **exit**.
- **Queues:** Job Queue (all processes in system) → Ready Queue (in-memory, ready to run — typically a linked list of PCBs) → Device Queues (one per I/O device, holds processes waiting for that specific device).
- **TRAP:** On a single-CPU system with n total processes, **at most 1 process can be Running** at any instant (max for Running = 1, regardless of n) — a classic "min/max processes in each state" numerical question. With P CPUs, at most **P** processes can be Running simultaneously.

2.4 Schedulers

Scheduler	Also called	Selects	Invocation frequency	Speed requirement
Long-Term	Job scheduler	Which jobs move from disk (job pool) into memory (Ready queue)	Rare (minutes)	Can be slow/thorough
Short-Term	CPU scheduler	Which Ready process gets the CPU next	Very frequent (~every 100 ms or less)	Must be very fast
Medium-Term	Swapper	Which processes get swapped OUT of memory (and back in later) to reduce/adjust the degree of multiprogramming	As needed (memory pressure)	Moderate

- **Degree of multiprogramming** = number of processes currently in memory; controlled by the **Long-Term scheduler**.
- **TRAP:** **Long-Term scheduler controls degree of multiprogramming** and must balance the **process mix** (CPU-bound vs I/O-bound) — too many I/O-bound processes starves the CPU scheduler (Ready queue empties), too many CPU-bound processes leaves I/O devices idle.

2.5 Dispatcher & Context Switch

- **Dispatcher:** the module that actually hands the CPU to the process chosen by the short-term scheduler — does context switch, switches to user mode, jumps to the resume location. Must be fast: the time it takes is **dispatch latency**.
- **Context Switch:** save the current process's state (into its PCB) + load the next process's saved state. **Pure overhead — no useful work is done during a context switch itself.**
- **CPU-bound process:** spends more time computing, generates I/O requests infrequently. **I/O-bound process:** spends more time waiting on I/O than computing.

Questions

Q1. A program becomes a process when:

A. It is written by a programmer B. It is compiled C. It is loaded into main memory and its PCB is created D. It is saved to disk

Answer: C. **Why:** A program is a passive file on disk; it becomes an active process only once loaded into memory with a PCB tracking its execution state.

Q2. Which of these is NOT typically stored in a Process Control Block?

A. Program counter B. CPU registers C. Source code text of the compiler used D. I/O status information

Answer: C. **Why:** The PCB stores runtime state (PC, registers, memory-management info, accounting, I/O status) — not unrelated details like which compiler produced the executable.

Q3. [Numerical/Trap] On a single-CPU system with n total processes, what is the maximum number of processes that can be in the Running state at once?

A. n B. $n-1$ C. 1 D. 0

Answer: C. **Why:** A single CPU can execute only one process at a time, regardless of how many total processes (n) exist.

Q4. Which scheduler is responsible for controlling the degree of multiprogramming?

A. Short-term scheduler B. Long-term scheduler C. Medium-term scheduler D. Dispatcher

Answer: B. **Why:** The long-term (job) scheduler decides which jobs are admitted from the disk pool into memory, directly setting how many processes reside in memory at once.

Q5. Why must the short-term (CPU) scheduler be much faster than the long-term scheduler?

A. It handles fewer processes B. It is invoked far more frequently (every ~ 100 ms or less) so any delay directly wastes CPU time C. It runs only once per session D. Speed doesn't matter for the short-term scheduler

Answer: B. **Why:** Since the CPU scheduler runs on the order of every 100 ms, any inefficiency in scheduler execution itself becomes significant overhead; the long-term scheduler runs far less often and can afford to be slower/more deliberate.

Q6. [Trap] What kind of process mix should the Long-Term Scheduler aim for?

A. All CPU-bound processes for maximum computation B. All I/O-bound processes for maximum device usage C. A balanced mix of CPU-bound and I/O-bound processes D. It doesn't matter what mix is chosen

Answer: C. **Why:** An all-I/O-bound mix starves the CPU scheduler (empty Ready queue); an all-CPU-bound mix leaves I/O devices idle — a balanced mix keeps both CPU and I/O subsystems utilized.

Q7. Context switch time is best described as:

A. Time during which useful process computation occurs B. Pure overhead — no useful work is done while switching C. Time spent only on I/O operations D. Time spent compiling the next process

Answer: B. **Why:** A context switch only saves/restores state; the CPU performs no productive computation for either process during that interval.

Q8. The module that performs the actual context switch and hands control to the process selected by the short-term scheduler is the:

A. Long-term scheduler B. Medium-term scheduler C. Dispatcher D. Job pool manager

Answer: C. **Why:** The dispatcher executes the mechanics of switching to the chosen process (context switch, mode switch, jump to resume point); the time this takes is the dispatch latency.

3. CPU Scheduling Algorithms

3.1 Preemptive vs Non-Preemptive

- **Non-preemptive:** once CPU is allocated, process keeps it until it terminates or voluntarily blocks for I/O.
- **Preemptive:** CPU can be forcibly taken away — on process termination, on voluntary I/O wait, when a higher-priority process becomes ready, or when the time quantum expires.

3.2 Scheduling Criteria & Core Formulas

Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
 Waiting Time (WT) = TAT - Burst Time (BT)
 Response Time = Time of FIRST CPU allocation - Arrival Time

- Goals: maximize CPU utilization & throughput; minimize TAT, WT, response time.
- **TRAP:** The scheduling algorithm does NOT change how long a process spends doing I/O — it only affects time spent **waiting in the ready queue**.

3.3 FCFS (First Come First Serve)

- Always **non-preemptive**; FIFO queue order by arrival time.
- **TRAP:** Suffers from **Convoy Effect** — short processes queue up behind one long process, inflating average waiting time.

Worked Example (reused across 3.3–3.6, 3.8 for comparison)

Processes: P1(AT=0,BT=5), P2(AT=1,BT=3), P3(AT=2,BT=8), P4(AT=3,BT=6)

FCFS order: P1, P2, P3, P4
 P1: 0-5 CT=5 TAT=5 WT=0
 P2: 5-8 CT=8 TAT=7 WT=4
 P3: 8-16 CT=16 TAT=14 WT=6
 P4: 16-22 CT=22 TAT=19 WT=13
 Avg TAT = (5+7+14+19)/4 = 11.25 Avg WT = (0+4+6+13)/4 = 5.75

3.4 SJF (non-preemptive) & SRTF (preemptive)

- **SJF:** among AVAILABLE processes, run the one with smallest total burst time; cannot be preempted once started, even if a shorter job arrives.
- **SRTF (Shortest Remaining Time First):** the preemptive version — re-evaluates on every new arrival; if the new arrival's burst < running process's REMAINING time, preempt.
- SRTF is **provably optimal** (minimum average waiting time of any scheduling algorithm) — but is NOT implementable in practice because burst time is never known in advance (only estimated, e.g. via exponential averaging $\tau(n+1) = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$).
- **TRAP:** SRTF/SJF risk **starvation** of long processes if short processes keep arriving.

Worked Example (same process set)

SJF (non-preemptive):
 P1: 0-5 (only one available) CT=5 TAT=5 WT=0
 P2: 5-8 (shortest of P2,P3,P4) CT=8 TAT=7 WT=4
 P4: 8-14 (shortest of P3,P4) CT=14 TAT=11 WT=5
 P3: 14-22 CT=22 TAT=20 WT=12
 Avg TAT = 43/4 = 10.75 Avg WT = 21/4 = 5.25

SRTF (preemptive):
 t=0: run P1(rem5)
 t=1: P2 arrives(rem3) < P1 rem(4) -> preempt, run P2
 t=2,3: P3,P4 arrive but P2's remaining (2, then 1) stays smallest -> P2 continues
 t=4: P2 completes (CT=4) TAT=3 WT=0
 t=4: shortest remaining = P1(4) -> run P1 to completion
 t=8: P1 completes (CT=8) TAT=8 WT=3
 t=8: shortest of P3(8),P4(6) = P4 -> run to completion
 t=14: P4 completes (CT=14) TAT=11 WT=5
 t=14-22: P3 runs alone CT=22 TAT=20 WT=12
 Avg TAT = 42/4 = 10.5 Avg WT = 20/4 = 5.0 <- lowest of all algorithms (optimal)

3.5 Priority Scheduling

- CPU given to highest-priority available process (convention for "highest" — lower or higher number — varies by system, always check the question's convention). Non-preemptive or preemptive variants exist, exactly like SJF/SRTF.
- TRAP:** Risk of **starvation** for low-priority processes — solved by **Aging** (gradually increase the priority of a process the longer it waits, e.g., +1 priority level every 10 minutes).

Worked Example (non-preemptive, LOWER number = higher priority)

```
P1(AT0,BT4,Pr2), P2(AT1,BT3,Pr1), P3(AT2,BT1,Pr3), P4(AT3,BT5,Pr4)
P1: 0-4 (only one available) CT=4 TAT=4 WT=0
P2: 4-7 (highest priority among P2,P3,P4) CT=7 TAT=6 WT=3
P3: 7-8 (higher priority than P4) CT=8 TAT=6 WT=5
P4: 8-13 CT=13 TAT=10 WT=5
Avg TAT = 26/4 = 6.5 Avg WT = 13/4 = 3.25
```

3.6 Round Robin (RR)

- Preemptive; ready queue is circular (FIFO); each process gets the CPU for at most **1 time quantum (TQ)**, then moves to the tail of the queue if not finished.
- If n processes and quantum q : each process waits **at most $(n-1) \times q$** time units between turns.

Time-quantum trade-off:

TQ too LARGE -> behaves like FCFS (worse response time)

TQ too SMALL -> approaches ideal "processor sharing," but context-switch overhead dominates -> lower CPU utilization

- TRAP:** Round Robin is designed to optimize **response time** for time-sharing systems — it does NOT generally minimize average waiting time (can even be worse than FCFS on some inputs, as the worked example shows).

Worked Example (same process set, quantum = 4)

```
t=0-4: P1 runs (rem 5-4=1); P2,P3,P4 arrive during this -> queue: [P2,P3,P4,P1(rem1)]
t=4-7: P2 runs to completion (rem3, finishes early) CT=7
t=7-11: P3 runs (rem 8-4=4) queue: [P4,P1(rem1),P3(rem4)]
t=11-15: P4 runs (rem 6-4=2) queue: [P1(rem1),P3(rem4),P4(rem2)]
t=15-16: P1 runs to completion (rem1) CT=16
t=16-20: P3 runs to completion (rem4) CT=20
t=20-22: P4 runs to completion (rem2) CT=22
TAT: P1=16,P2=6,P3=18,P4=19 Avg TAT=59/4=14.75
WT: P1=11,P2=3,P3=10,P4=13 Avg WT=37/4=9.25 (worse than FCFS here!)
```

3.7 LJF / LRTF and HRRN

- LJF (Longest Job First) / LRTF (Longest Remaining Time First, preemptive)** — opposite of SJF/SRTF; included mainly for contrast, rarely optimal, generally poor average waiting time (favors long jobs, starves short ones).
- HRRN (Highest Response Ratio Next)** — non-preemptive; solves SJF's starvation problem by factoring in wait time:

Response Ratio = (Waiting Time + Burst Time) / Burst Time

Highest response ratio runs next — a process's ratio grows the longer it waits, guaranteeing it eventually gets scheduled (no starvation), while still favoring short jobs early on.

Worked Example (same process set)

```
t=5: candidates P2(RR=(4+3)/3=2.33), P3(RR=(3+8)/8=1.375), P4(RR=(2+6)/6=1.333) -> run P2
t=8: candidates P3(RR=(6+8)/8=1.75), P4(RR=(5+6)/6=1.833) -> run P4
t=14: only P3 left -> run P3
Result identical to SJF-NP for this data set: Avg TAT=10.75, Avg WT=5.25
```

3.8 Multilevel Queue vs Multilevel Feedback Queue

Feature	Multilevel Queue	Multilevel Feedback Queue
Queue assignment	Permanent (fixed at entry, e.g. by process type)	Dynamic — process can move between queues
Flexibility	Low (simple, low overhead)	High (most general/configurable CPU scheduling algorithm)
Handles aging?	No	Yes — long-waiting process promoted to a higher-priority queue
Inter-queue scheduling	Fixed priority (higher queue starves lower) or time-sliced % of CPU	Defined by scheduler's demotion/promotion rules

- Multilevel Feedback Queue is defined by: **(1)** number of queues, **(2)** scheduling algorithm per queue, **(3)** rule to upgrade a process, **(4)** rule to demote a process, **(5)** rule for which queue a new process enters.
- TRAP:** Multilevel Queue = fixed/rigid queue membership; Multilevel FEEDBACK Queue = processes migrate — do not confuse the two.

3.9 Summary Comparison

Algorithm	Preemptive?	Starvation risk?	Optimal avg WT?
FCFS	No	No (but Convoy Effect)	No
SJF	No	Yes (long jobs)	Best among non-preemptive
SRTF	Yes	Yes (long jobs)	Yes — optimal overall
Priority	Either	Yes (low priority) — fixed by Aging	No
Round Robin	Yes	No	No (optimizes response time instead)
HRRN	No	No (response ratio prevents it)	Close to SJF, no starvation
Multilevel Feedback Queue	Yes	No (aging built in)	Configurable / most general

Questions

Q1. [Numerical] For P1(AT=0,BT=5), P2(AT=1,BT=3), P3(AT=2,BT=8), P4(AT=3,BT=6) under FCFS, what is the average waiting time?

A. 4.25 B. 5.0 C. 5.75 D. 6.5

Answer: C. **Why:** WTs are 0,4,6,13 → average = $23/4 = 5.75$ (see worked example, §3.3).

Q2. [Trap] Which statement about FCFS is correct?

A. It is always preemptive B. It never suffers from convoy effect C. It is always non-preemptive and can suffer from the convoy effect D. It requires knowing burst time in advance

Answer: C. **Why:** FCFS is purely arrival-order based and non-preemptive; a long process at the front delays all shorter ones behind it (convoy effect).

Q3. Why is SRTF considered optimal but practically unusable?

A. It has high context-switch overhead only B. It provably minimizes average waiting time but requires knowing future burst times, which is generally impossible C. It always causes deadlock D. It cannot be preempted

Answer: B. **Why:** SRTF/SJF need the exact CPU burst length in advance; real systems can only estimate it, so pure SRTF cannot actually be implemented.

Q4. [Numerical] Using the same process set as the worked examples, what is the average turnaround time under SRTF?

A. 10.5 B. 10.75 C. 11.25 D. 14.75

Answer: A. **Why:** TATs are 8,3,20,11 → average = $42/4 = 10.5$ (lowest among all algorithms compared, confirming SRTF's optimality).

Q5. [Trap] A low-priority process never gets the CPU because higher-priority processes keep arriving. Which technique fixes this?

A. Round Robin quantum reduction B. Aging — gradually increasing the waiting process's priority over time C. Switching to FCFS D. Increasing the time quantum

Answer: B. **Why:** Aging directly counters starvation by boosting priority the longer a process waits, ensuring it eventually becomes highest priority.

Q6. [Numerical] In Round Robin with $n=5$ ready processes and time quantum $q=3$, what is the maximum time any process waits between two of its turns?

A. 3 B. 12 C. 15 D. 5

Answer: B. **Why:** Max wait = $(n-1) \times q = 4 \times 3 = 12$.

Q7. [Trap] What happens to Round Robin's behavior if the time quantum is set extremely large?

A. It behaves like SRTF B. It behaves like FCFS C. It becomes Priority Scheduling D. It causes immediate deadlock

Answer: B. **Why:** If the quantum exceeds every process's burst time, each process finishes in its first turn — identical to FCFS ordering.

Q8. HRRN's Response Ratio formula is:

A. Burst Time / Waiting Time B. (Waiting Time + Burst Time) / Burst Time C. Waiting Time – Burst Time D. Burst Time / (Waiting Time + Burst Time)

Answer: B. **Why:** $RR = (W+S)/S$ — this rises over time for any waiting process (even a long one), preventing indefinite starvation while still favoring short jobs.

Q9. [Trap] What is the key structural difference between Multilevel Queue and Multilevel Feedback Queue scheduling?

A. There is no difference B. Multilevel Queue allows movement between queues; Feedback does not C. Multilevel Feedback Queue allows a process to move between queues (e.g. via aging); plain Multilevel Queue assigns processes permanently D. Multilevel Queue is always preemptive, Feedback is not

Answer: C. **Why:** The defining feature of the "feedback" variant is dynamic reassignment between queues based on observed behavior/waiting time; plain multilevel queue is static once a process is classified.

Q10. [Numerical] For P1(AT0,BT4,Pr2), P2(AT1,BT3,Pr1), P3(AT2,BT1,Pr3), P4(AT3,BT5,Pr4) under non-preemptive priority scheduling (lower number = higher priority), what is P3's waiting time?

A. 1 B. 5 C. 6 D. 3

Answer: B. **Why:** P3 runs from $t=7$ to $t=8$ ($CT=8$); $TAT = 8 - 2 = 6$; $WT = TAT - BT = 6 - 1 = 5$.

Q11. Which CPU scheduling algorithm is described as "the most general," configurable via number of queues, per-queue algorithm, and promotion/demotion rules?

A. Priority Scheduling B. Round Robin C. Multilevel Feedback Queue Scheduling D. HRRN

Answer: C. **Why:** Multilevel Feedback Queue can be tuned via 5 defining parameters, making it flexible enough to emulate many other scheduling behaviors — at the cost of high design complexity.

4. Process Synchronization — Critical Section & Semaphores

4.1 Race Condition & Critical Section

- **Race condition:** final output depends on the (unpredictable) relative execution order/interleaving of concurrent processes accessing shared data.
- **General process structure:** Entry Section (request permission) → **Critical Section** (accesses shared resource) → Exit Section (release) → Remainder Section.

4.2 The 3 Critical-Section Requirements

1. **Mutual Exclusion:** at most one process may be inside its critical section at any instant.
2. **Progress:** if no process is in its CS and some wish to enter, only processes NOT in their remainder section may participate in deciding who enters next — this decision cannot be postponed indefinitely (no deadlock in the decision itself).
3. **Bounded Waiting:** there is a fixed limit on how many times other processes may enter their CS after a process has requested entry, before that process is granted entry (no indefinite postponement / no starvation).
 - **TRAP: Mutual Exclusion and Progress are mandatory; Bounded Waiting is often treated as the optional/stronger criterion** — a valid solution can satisfy ME+Progress but still risk starvation if Bounded Waiting isn't met.

4.3 Two-Process Software Solutions

Approach	Mutual Exclusion?	Progress?	Issue
Boolean <code>turn</code> variable (strict alternation)	Yes	No	Forces strict turn-by-turn access even if the other process doesn't want to enter — violates Progress
Boolean <code>flag[]</code> array (intent flags)	Yes	No	Both processes can set their flag simultaneously and wait forever on each other → deadlock
Peterson's Solution (turn + <code>flag[]</code> combined)	Yes	Yes	Satisfies all 3 properties, but only for exactly 2 processes ; not guaranteed on modern multiprocessor architectures (relies on memory-ordering assumptions)

- **TRAP:** Peterson's Solution is a classic *software-only* 2-process solution — it does NOT generalize directly to N processes (that's what semaphores are for).

4.4 Semaphores

- A semaphore S is an integer accessed only via two **atomic** operations:

```
wait(S) / P(S):  while (S <= 0) ;  S--;
signal(S) / V(S): S++;
```

- Semaphore-based CS solution: initialize `S=1`, then `wait(S)` before CS, `signal(S)` after CS — generalizes to **N processes** (unlike Peterson's).
- Semaphores guarantee **Mutual Exclusion and Progress**, but the basic busy-wait version does **NOT guarantee Bounded Waiting** by itself.
- **Binary semaphore:** value restricted to {0,1} — behaves like a mutex lock.
- **Counting semaphore:** unrestricted integer range — models a pool of N identical resource instances; initialized to N, each `wait()` claims one instance, each `signal()` releases one.
- **TRAP:** Ordering multiple `wait()` calls on different semaphores **inconsistently across processes** (e.g., P: `wait(S1)` then `wait(S2)`; Q: `wait(S2)` then `wait(S1)`) risks **deadlock** — always acquire multiple semaphores in the same global order across all processes.

4.5 Classical Synchronization Problems

Problem	Core requirement	Key semaphores/variables
Producer-Consumer (bounded buffer, size n)	Producer must not overflow; consumer must not underflow; buffer access is mutually exclusive	$S=1$ (mutex for buffer), $E=n$ (count of empty slots), $F=0$ (count of filled slots) — producer: <code>wait(E), wait(S), ..., signal(S), signal(F)</code> ; consumer: <code>wait(F), wait(S), ..., signal(S), signal(E)</code>
Readers-Writers (readers-priority version)	Multiple readers may read concurrently; a writer needs exclusive access; no reader/writer runs while a writer is writing	<code>mutex=1</code> (protects <code>readcount</code>), <code>wrt=1</code> (writer's exclusive lock); first reader does <code>wait(wrt)</code> , last reader does <code>signal(wrt)</code> — <code>readcount</code> tracks active readers
Dining Philosophers (5 philosophers, 5 chopsticks)	Each philosopher needs both adjacent chopsticks to eat; naive "pick left then right" for everyone → deadlock (all pick up left simultaneously)	Semaphore array <code>chopstick[5]</code> ; deadlock fixes: (a) allow at most N-1 (4) philosophers to sit at once, (b) pick up both chopsticks only inside a critical section (atomically), (c) asymmetric solution — odd-numbered philosopher picks left-then-right, even-numbered picks right-then-left

- **TRAP:** The naive Dining Philosophers solution (everyone picks up left chopstick first) is NOT deadlock-free — this is the whole point of the problem; a valid fix (limit to $N-1$ diners, or asymmetric pickup order) must be stated explicitly.

4.6 Semaphore Implementation: Busy-Waiting vs Block/Wakeup

- **Busy-wait implementation** (the simple `while(S<=0);` loop) wastes CPU cycles while waiting — bad on a single-processor system.
- **Block-and-wakeup implementation:** if `wait()` finds $S \leq 0$, the process calls `block()` — it's added to the semaphore's waiting list and suspended (Ready→Waiting), and the CPU is given to another process. `signal()` calls `wakeup(P)` on a waiting process, moving it back to Ready.
- **TRAP:** In the block/wakeup implementation, the semaphore's integer **value can go negative** — its magnitude then equals the number of processes currently waiting on it (this differs from the classical busy-wait definition where S never goes below 0).

4.7 Hardware Synchronization Primitives

- **Disable Interrupts:** simplest single-processor solution (disable interrupts in the entry section, re-enable in exit section) — prevents any context switch during the CS. **TRAP:** does **not scale to multiprocessor systems** (must message every processor to disable interrupts — slow, and doesn't stop a process running on ANOTHER CPU from entering its own CS simultaneously).
- **Test-and-Set (TAS):** an atomic hardware instruction that reads and sets a boolean in one indivisible step:

```
Boolean test_and_set(Boolean *target) {
    Boolean rv = *target;
    *target = true;
    return rv;
}
```

Used as a spinlock: `while(test_and_set(&lock));` then CS, then `lock=false;`. Works correctly on **multiprocessor** systems (unlike disabling interrupts) since the atomicity is a hardware guarantee on the memory word itself, not on interrupt delivery.

Questions

Q1. Which of the 3 critical-section requirements is generally considered optional, whose absence risks starvation but not necessarily an invalid solution?

A. Mutual Exclusion B. Progress C. Bounded Waiting D. Atomicity

Answer: C. **Why:** Mutual Exclusion and Progress are mandatory; Bounded Waiting's absence specifically risks (but doesn't guarantee) starvation of some process.

Q2. [Trap] The two-process solution using a single Boolean `turn` variable with strict alternation fails to satisfy:

A. Mutual Exclusion B. Progress (forces alternation even if one process doesn't want to enter) C. Both ME and Progress D. It satisfies all three properties

Answer: B. **Why:** It achieves ME but fails Progress since a process must wait its turn even when the other process has no interest in entering its CS.

Q3. Peterson's Solution is valid for:

A. Any number of processes B. Exactly 2 processes C. Exactly 3 processes D. Only single-processor hardware with disabled interrupts

Answer: B. **Why:** Peterson's Solution is specifically a 2-process software algorithm; semaphores are needed to generalize to N processes.

Q4. A counting semaphore initialized to 5 models:

A. A binary lock B. A resource pool with 5 identical instances available C. A queue of exactly 5 waiting processes D. 5 separate critical sections

Answer: B. **Why:** Counting semaphores track available instances of a resource type; each `wait()` claims one (decrementing), each `signal()` returns one (incrementing).

Q5. In the Readers-Writers (readers-priority) solution, which reader is responsible for acquiring the `wrt` lock?

A. Every reader, individually B. Only the FIRST reader to arrive when `readcount` goes from 0 to 1 C. Only the last reader D. No reader ever touches `wrt`

Answer: B. **Why:** Only the first reader (transitioning `readcount` 0→1) needs to lock out writers via `wait(wrt)`; subsequent readers just increment `readcount` under mutex. The LAST reader (`readcount`→0) calls `signal(wrt)`.

Q6. [Trap] Why is the naive Dining Philosophers solution (every philosopher picks up left chopstick, then right) invalid?

A. It violates mutual exclusion on each chopstick B. All 5 philosophers can simultaneously pick up their left chopstick, then all wait forever for their right — deadlock C. It runs out of memory D. It doesn't use semaphores at all

Answer: B. **Why:** Symmetric pickup order allows a circular-wait deadlock; fixes include limiting concurrent diners to $N-1$ or using an asymmetric (odd/even) pickup order.

Q7. [Trap] In the block-and-wakeup implementation of a semaphore, a semaphore value of -3 means:

A. An implementation error occurred B. 3 processes are currently blocked/waiting on this semaphore C. The semaphore has 3 available resources D. 3 processes have exited

Answer: B. **Why:** Under block/wakeup, value can go negative; its absolute value equals the number of processes queued waiting on that semaphore.

Q8. [Trap] Why doesn't disabling interrupts work as a general critical-section solution on multiprocessor systems?

A. Interrupts cannot be disabled on any modern CPU B. It only blocks interrupts on the local processor, so a process on another CPU can still enter the same critical section; and multiprocessor-wide interrupt disabling is slow C. It causes immediate system crash D. It works perfectly fine on multiprocessors

Answer: B. **Why:** Disabling interrupts is inherently a single-processor mutual-exclusion technique; message-passing to disable interrupts across all CPUs is costly and doesn't provide atomicity on shared memory.

Q9. The Test-and-Set instruction's key guarantee that makes it useful for synchronization is:

A. It is always fast B. It reads and modifies a memory location as one atomic (indivisible) hardware operation C. It disables all interrupts automatically D. It only works with binary semaphores

Answer: B. **Why:** Atomicity means even if two CPUs execute test-and-set on the same word "simultaneously," the hardware serializes them — one gets the old (false) value and proceeds, the other gets true and spins.

Q10. Which classical problem's semaphore solution uses the pair of semaphores E (count of empty slots) and F (count of filled slots)?

A. Readers-Writers B. Dining Philosophers C. Producer-Consumer (bounded buffer) D. Peterson's Solution

Answer: C. **Why:** Producer waits on E (space available) and signals F (item added); Consumer waits on F (item available) and signals E (space freed) — plus a mutex S for the buffer itself.

5. Threads & Multithreading Models

5.1 Process vs Thread

- **Process** = heavyweight, has its own independent memory space (code, data, heap, stack, PCB, resources).
- **Thread** = "lightweight process" — a basic unit of CPU utilization within a process. Has its own program counter, stack, register set, and thread ID, but **shares** code, data segment, and OS resources (open files, etc.) with sibling threads in the same process.
- **TRAP:** Threads within one process share the heap/global data — this is exactly why thread synchronization (mutexes/semaphores) is needed even though process-level protection (separate address spaces) already isolates *processes* from each other.

5.2 fork() System Call

- `fork()` creates a **child process** as a near-exact copy (duplicate image) of the calling (parent) process.
- Return value convention (classic exam fact):

```
fork() returns 0           -> in the CHILD process
fork() returns child's PID -> in the PARENT process (a positive integer)
fork() returns negative    -> on failure (no child created)
```

- Because both parent and child continue executing from the point right after `fork()`, programmer code typically branches on the return value to give parent and child different behavior.
- **Numerical rule:** n calls to `fork()` (e.g., in a loop or sequential calls, with no conditional branching) create $2^n - 1$ new processes in total (2^n processes exist overall including the original).
- **TRAP:** Don't confuse this with "n forks create n processes" — each fork doubles the process count from that point onward, giving exponential (2^n) growth, not linear.

5.3 Why Multithreading

- **Responsiveness:** one thread can keep the UI/interaction responsive while another blocks (e.g., disk I/O).
- **Resource sharing:** threads share the parent's memory/resources — cheaper than duplicating them per process.
- **Economy:** creating/context-switching a thread is much cheaper than a full process (no new address space needed).
- **Scalability:** threads can run in parallel on multicore/multiprocessor systems.

5.4 Multithreading Models

Threads exist at two levels: **User-level threads** (managed entirely in user space, no kernel awareness) and **Kernel-level threads** (managed/scheduled by the OS kernel directly).

Model	Mapping	Key Strength	Key Weakness
Many-to-One	Many user threads → 1 kernel thread	Fast, cheap thread management (no kernel calls)	One blocking system call blocks the entire process; no true parallelism (only 1 kernel thread = only 1 CPU used ever)
One-to-One	Each user thread → its own kernel thread	True parallelism across CPUs; a blocking call in one thread doesn't block others	Higher overhead — creating a kernel thread per user thread is costly; most OSes cap the number allowed
Many-to-Many	Many user threads → smaller/equal number of kernel threads	Combines both benefits: unlimited user threads + no full-process blocking + true parallelism	Most complex to implement

- **TRAP:** **Many-to-One** is the model where a single blocking syscall stalls every thread in the process — this is its single biggest exam-tested weakness. **One-to-One** (used by modern Linux and Windows) avoids that but at higher per-thread overhead.

Questions

Q1. [Trap] In a Unix-like system, what does `fork()` return inside the newly created child process?

A. The child's own PID B. 0 C. The parent's PID D. -1

Answer: B. **Why:** `fork()` returns 0 in the child; it returns the child's PID to the parent, and a negative value only on failure.

Q2. [Numerical] A program calls `fork()` three times in sequence (no conditional logic, so every resulting process also executes the remaining `fork()` calls). How many processes exist in total after all forks complete?

A. 3 B. 6 C. 7 D. 8

Answer: D. **Why:** n forks give 2^n total processes = $2^3 = 8$ (i.e., $2^n - 1 = 7$ new processes plus the original 1).

Q3. Which multithreading model suffers from the problem that a single blocking system call halts every thread in the process?

A. One-to-One B. Many-to-Many C. Many-to-One D. None of these

Answer: C. Why: With Many-to-One, all user threads map onto a single kernel thread, so if that one kernel thread blocks (e.g., on I/O), the entire process — all its threads — is blocked.

Q4. Which multithreading model is used by modern Linux and Windows, giving true parallel execution across CPUs at the cost of per-thread kernel overhead?

A. Many-to-One B. One-to-One C. Many-to-Many D. Two-to-One

Answer: B. Why: One-to-One maps each user thread directly to its own kernel thread, enabling genuine multi-CPU parallelism, though creating many kernel threads has more overhead than the other models.

Q5. Which of the following is shared between threads of the same process, but NOT between separate processes?

A. Program counter B. Stack C. Code and data segment D. Register set

Answer: C. Why: Threads of one process share the code/data segment and other resources; each thread still keeps its own PC, stack, and registers, which is what makes it independently schedulable.

Q6. [Conceptual] Why is creating a new thread generally cheaper than creating a new process?

A. Threads don't need a program counter B. A thread reuses the parent process's address space/resources instead of duplicating them C. Threads cannot be scheduled by the OS D. Threads always run on a separate CPU

Answer: B. Why: Since threads share the code, data, and resources of the owning process, thread creation avoids the overhead of setting up an entirely new address space, unlike process creation.

6. Deadlock — Prevention, Avoidance, Detection

6.1 Basics

- **Deadlock:** every process in a set is waiting for an event (resource release) that can only be caused by another process in that same set — none can ever proceed.
- **TRAP: Starvation = long/indefinite waiting** (may eventually get served); **Deadlock = infinite waiting** (can never be served without external intervention). Starvation does not imply deadlock, but deadlock does imply (permanent) starvation of the involved processes.
- Normal resource usage cycle: **Request** → **Use** → **Release**.

6.2 The 4 Necessary Conditions (ALL must hold simultaneously)

1. **Mutual Exclusion** — at least one resource is held in non-sharable mode (only one process can use it at a time).
2. **Hold and Wait** — a process holds at least one resource while waiting to acquire additional resources held by others.
3. **No Preemption** — a resource can only be released voluntarily by the process holding it (never forcibly taken).
4. **Circular Wait** — a cycle of processes $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_0$ exists, each waiting for a resource held by the next.

TRAP: Breaking even **ONE** of these four conditions prevents deadlock — this single fact is the entire basis of every deadlock-prevention strategy below.

6.3 Deadlock Handling Approaches — Overview

Approach	Idea
Prevention	Structurally deny one of the 4 conditions so deadlock is impossible
Avoidance	Allow all 4 conditions to be possible, but dynamically check every resource request to avoid entering an unsafe state (Banker's Algorithm)
Detection & Recovery	Let deadlock happen, periodically detect it, then recover
Ignorance (Ostrich Algorithm)	Pretend deadlocks don't happen — used by most general-purpose OSes (Linux/Windows) since deadlocks are rare in practice and prevention/avoidance costs (reduced concurrency/throughput) aren't worth it

6.4 Deadlock Prevention (deny one condition)

- **Mutual Exclusion:** generally CANNOT be denied — some resources (printer, etc.) are intrinsically non-sharable by hardware nature.
- **Hold and Wait:** deny by requiring a process to request ALL its resources upfront before running (conservative), or require releasing everything held before requesting more.
- **No Preemption:** deny by allowing forcible preemption — if a waiting process requests unavailable resources, preempt resources already held by OTHER waiting processes instead.
- **Circular Wait:** deny by imposing a **total ordering** on resource types — a process may only request resources in strictly increasing order (must release higher-numbered resources before requesting a lower-numbered one).
- Drawback of prevention overall: low resource utilization and reduced system throughput/concurrency (overly restrictive).

6.5 Deadlock Avoidance — Banker's Algorithm

Requires advance knowledge of each process's **maximum** possible resource demand.

```
Data structures (n = #processes, m = #resource types):
Available[m]      - available instances of each resource type
Max[n][m]         - max demand of each process
Allocation[n][m]  - currently allocated to each process
Need[n][m] = Max[n][m] - Allocation[n][m]  (remaining demand)
```

Safety Algorithm (is the current state safe?):

```
1. Work = Available; Finish[i] = false for all i
2. Find i where Finish[i]==false AND Need[i] <= Work
   - if none exists -> go to step 4
3. Work = Work + Allocation[i]; Finish[i] = true; go to step 2
4. If Finish[i]==true for all i -> state is SAFE (the order found = safe sequence)
```

Resource-Request Algorithm (grant a request only if it keeps the system safe):

1. If Request[i] > Need[i] -> error (exceeds max claim)
2. If Request[i] > Available -> P_i must wait (not enough resources right now)
3. Pretend to allocate: Available -= Request; Allocation[i] += Request; Need[i] -= Request
 - Run Safety Algorithm on this new state.
 - If SAFE -> grant the request for real.
 - If UNSAFE -> rollback the pretend-allocation; P_i must wait.

Worked Example (verified)

Resources: E, F, G	Available = (3, 3, 0)	
Max	Allocation	Need = Max - Allocation
P0 (4, 3, 1)	(1, 0, 1)	(3, 3, 0)
P1 (2, 1, 4)	(1, 1, 2)	(1, 0, 2)
P2 (1, 3, 3)	(1, 0, 3)	(0, 3, 0)
P3 (5, 4, 1)	(2, 0, 0)	(3, 4, 1)

Run safety algorithm: Work=(3,3,0).

- P0: Need(3,3,0) ≤ Work(3,3,0) ✓ → Work += Alloc(P0) = (4,3,1)
 - P2: Need(0,3,0) ≤ (4,3,1) ✓ → Work += Alloc(P2) = (5,3,4)
 - P1: Need(1,0,2) ≤ (5,3,4) ✓ → Work += Alloc(P1) = (6,4,6)
 - P3: Need(3,4,1) ≤ (6,4,6) ✓ → Work += Alloc(P3) = (8,4,6)
- All Finish=true → **Safe sequence = P0, P2, P1, P3** → system is in a safe state.

6.5b Resource Allocation Graph (RAG)

- Nodes: **processes** (circles) + **resource types** (rectangles, one dot per instance).
- **Request edge** P_i → R_j (waiting for an instance). **Assignment edge** R_j → P_i (already holds an instance).
- **TRAP:** A cycle in the RAG is **necessary but NOT sufficient** for deadlock when a resource type has **multiple instances** (the cycle might resolve if another instance frees up). A cycle is **necessary AND sufficient** for deadlock **only when every resource type has a single instance**.

6.6 Deadlock Detection & Recovery

- Detection algorithm = essentially the Safety Algorithm run periodically, but using only **Allocation** and **Request** (no Max/Need needed, since we're not avoiding — just checking if the CURRENT state is already deadlocked).
- Invocation strategy: run periodically (e.g., hourly) or reactively (e.g., when CPU utilization drops suggesting processes are stuck waiting).
- **Recovery — Process Termination:** Abort ALL deadlocked processes at once (guaranteed but wastes all their computation) vs abort ONE at a time + re-run detection after each (more careful, but repeated detection overhead).
- **Recovery — Resource Preemption:** pick a victim, preempt its resources, roll it back to a safe state. Victim selection weighs cost factors: priority, computation time invested, resources held, resources still needed. Must track rollback counts per process to avoid **starving the same victim repeatedly**.

Questions

Q1. Which of the following is NOT one of the four necessary conditions for deadlock?

A. Mutual Exclusion B. Hold and Wait C. Starvation D. Circular Wait

Answer: C. Why: Starvation is a separate phenomenon (indefinite but not necessarily infinite waiting); the four deadlock conditions are Mutual Exclusion, Hold-and-Wait, No Preemption, and Circular Wait.

Q2. [Trap] Deadlock prevention typically cannot deny which condition, since it is often an intrinsic hardware property of the resource?

A. Hold and Wait B. Mutual Exclusion C. Circular Wait D. No Preemption

Answer: B. Why: Many resources (e.g., a printer) are inherently non-sharable, so Mutual Exclusion generally cannot be eliminated as a prevention strategy.

Q3. [Numerical] Using the Banker's Algorithm example: Available=(3,3,0); Need(P0)=(3,3,0), Need(P1)=(1,0,2), Need(P2)=(0,3,0), Need(P3)=(3,4,1). Which process(es) CANNOT be satisfied by the initial Available=(3,3,0) alone (i.e., must wait for others to release resources first)?

A. Only P3 B. P1 and P3 C. Only P1 D. None — all can be satisfied immediately

Answer: B. Why: P1's Need (1,0,2) needs 2 units of G but Available has 0 of G; P3's Need (3,4,1) needs 4 units of F but Available has only 3 — both must wait for P0 or P2 to finish first and release resources, which the full safe sequence (P0, P2, P1, P3) confirms is possible.

Q4. A cycle exists in a Resource Allocation Graph where every resource type has exactly one instance. What can be concluded?

A. The system is definitely NOT deadlocked B. The system is definitely deadlocked C. Nothing can be concluded D. More information about process priority is needed

Answer: B. Why: With single-instance resource types, a cycle in the RAG is both necessary AND sufficient for deadlock.

Q5. [Trap] A cycle exists in a Resource Allocation Graph where at least one resource type has multiple instances. What can be concluded?

A. The system is definitely deadlocked B. The system is definitely safe C. A cycle is necessary but not sufficient — deadlock is only possible, not guaranteed D. RAGs cannot be drawn for multi-instance resources

Answer: C. **Why:** With multi-instance resource types, another instance might free up and break the apparent cycle's hold, so a cycle alone doesn't guarantee deadlock.

Q6. Which deadlock-handling approach is used by most general-purpose operating systems like Linux and Windows?

A. Prevention B. Avoidance (Banker's Algorithm) C. Detection and Recovery D. Ignorance (Ostrich Algorithm)

Answer: D. **Why:** Since deadlocks are rare in practice and prevention/avoidance impose significant performance/concurrency costs, most general-purpose OSes simply ignore the possibility and rely on manual/general recovery if it ever occurs.

Q7. In deadlock recovery via resource preemption, why must the "number of rollbacks" be tracked per process?

A. To calculate CPU burst time B. To prevent the same process from being repeatedly chosen as a victim (starvation) C. To determine the process's priority D. It is not necessary to track this

Answer: B. **Why:** Without tracking rollback counts, a low-cost victim might be picked every single time, indefinitely starving that one process even though the overall deadlock keeps recurring for others.

7. Memory Management — Contiguous Allocation, Paging, Segmentation

7.1 Memory Hierarchy & Locality of Reference

- Hierarchy (fastest/smallest/costliest → slowest/largest/cheapest): **Registers** → **Cache (L1/L2/L3)** → **Main Memory (RAM)** → **Secondary/Auxiliary Memory (disk)**.
- **Locality of Reference** is why caching/paging work at all — predicts what to prefetch.
- **Temporal locality**: a recently used item will likely be used again soon (basis of **LRU**).
- **Spatial locality**: items near a used address will likely be used soon too (basis of block/page prefetching).
- OS's memory duties: logical → physical address translation, deciding what to swap in/out, tracking free/used memory.

7.2 Contiguous Allocation

- Process must be loaded **entirely** and **contiguously** into main memory.
- **MMU (Memory Management Unit)**: hardware that translates logical → physical address using a **relocation register** (base address, added to every logical address) and a **limit register** (max allowed logical address — if logical address $\geq$ limit, illegal access is trapped by the OS).

Physical Address = Logical Address + Relocation Register (valid only if Logical Address < Limit Register)

- **Partitioning**: Fixed-size partitioning (simple, but internal fragmentation) vs Variable-size partitioning (exact fit per request, but external fragmentation accumulates).

Space Allocation Strategies

Strategy	Rule	Best for	Weakness
First Fit	First partition big enough	Speed	Poor packing over time
Best Fit	Smallest partition big enough	Minimizes leftover per allocation	Slow (full scan); leaves tiny unusable slivers
Worst Fit	Largest available partition	Leaves large reusable leftover in variable partitioning	Wastes large amounts in fixed partitioning
Next Fit	Like First Fit, but resumes scan from last allocation point	Avoids re-scanning from start	Similar issues to First Fit

- **External fragmentation**: enough TOTAL free memory exists, but it's scattered in pieces too small individually — fixed by **compaction** (costly, requires stopping/relocating processes) or moving to **non-contiguous allocation**.
- **Internal fragmentation**: allocated partition $\geq$ request size; the leftover INSIDE the allocated block is wasted (inherent to fixed-size schemes).
- **TRAP**: External fragmentation $\Rightarrow$ variable-size/contiguous schemes; Internal fragmentation $\Rightarrow$ fixed-size schemes (including paging, Ch 7.3). Don't swap these.

7.3 Paging (Non-Contiguous Allocation)

- Physical memory → fixed-size **frames**; logical/process memory → same-size **pages**. Page size = Frame size. Eliminates external fragmentation (any free frame fits any page) but keeps internal fragmentation (last page may be partly empty).
- **Address translation**:

Logical Address = (Page Number p , Offset d)
 - if Page size = 2^d bytes and Logical Address Space = 2^m , then p occupies $(m - d)$ bits
 Physical Address = (Frame Number f , Offset d) [offset d is IDENTICAL on both sides]
 Page Table: indexed by $p \rightarrow$ gives f

Worked Example

Page size = 1 KB (2^{10}), Logical Address = 5000.
 Page number = $\text{floor}(5000 / 1024) = 4$ (page 4 covers bytes 4096–5119); Offset = $5000 - 4096 = 904$.
 If page table says page 4 → frame 7: Physical Address = $7 \times 1024 + 904 = 8072$.

- **Page table** is a data structure (not hardware) stored in main memory; one per process; entries = number of pages; each entry stores a frame number.
- Disadvantage: naive paging needs **2 memory accesses per reference** (1 for page table, 1 for actual data) — solved by the **TLB**.

7.4 TLB (Translation Lookaside Buffer)

- Small, fast, **associative** (parallel-search) hardware cache of recent (page→frame) mappings.
- **TLB hit** → 1 memory access needed. **TLB miss** → page table lookup + memory access (2 accesses total, or 3 if you count the TLB check itself, depending on convention — use the standard 2-vs-1 split for EAT).

Effective Access Time (EAT) = HitRatio × (TLB time + Memory time) + MissRatio × (TLB time + Memory time + Memory time)

Worked Example

TLB access = 20 ns, Memory access = 100 ns, Hit ratio = 80%.

EAT = $0.8 \times (20 + 100) + 0.2 \times (20 + 100 + 100) = 0.8 \times 120 + 0.2 \times 220 = 96 + 44 = \mathbf{140 \text{ ns}}$.

- TLB holds entries for ONE process context at a time in the simplest design → frequent context switches force **TLB flush** (costly); mitigated by wiring down kernel entries or tagging entries with a process/address-space ID.

7.5 Multilevel (Hierarchical) Paging

- Modern address spaces (32–64 bit) make a single-level page table huge (millions of entries, too big for one frame).
- Fix: **page the page table itself** — a two-level (or more) scheme with an outer page table pointing to inner page table pages. Trade-off: MORE memory accesses per translation (one per level + 1 for data), but manageable table size.
- Page-size trade-off: bigger pages → smaller page table but MORE internal fragmentation; smaller pages → less internal fragmentation but LARGER page table. Optimal page size balances both costs.
- **Inverted Page Table**: one entry PER PHYSICAL FRAME (not per logical page) storing which process+page occupies it → table size = number of frames (not pages), saving huge amounts of memory when address spaces are large, at the cost of slower lookups (needs associative/hashed search).

7.6 Segmentation

- Supports the **user's logical view** of a program (code segment, stack segment, data segment — variable-sized, meaningful units) — contrast directly with paging's **physical**, fixed-size, programmer-invisible division.
- **TRAP: Segmentation = logical/variable-size (user view). Paging = physical/fixed-size (system view)**. This exact contrast is a classic recall question.
- Logical address = (segment number, offset). Segment table entry = (base, limit) per segment; offset must be < limit or it traps to the OS.

Physical Address = Segment Base + Offset (valid only if $0 \leq \text{Offset} < \text{Segment Limit}$)

- Segmentation suffers **external fragmentation** (segments are variable-sized, like variable contiguous allocation).

Worked Example (Segment/Limit Check)

Limit/Relocation table: P0(limit=500, reloc=1200), P1(limit=275, reloc=550), P2(limit=212, reloc=880), P3(limit=420, reloc=1400), P4(limit=118, reloc=200).

Requests: P0 offset 450 → $450 < 500$ valid → PA = $1200 + 450 = \mathbf{1650}$.

P1 offset 300 → $300 \geq 275$ → **illegal (trap)**.

P2 offset 210 → $210 < 212$ valid → PA = $880 + 210 = \mathbf{1090}$.

P3 offset 450 → $450 \geq 420$ → **illegal (trap)**.

P4 offset 80 → $80 < 118$ valid → PA = $200 + 80 = \mathbf{280}$.

7.7 Segmentation with Paging (Hybrid)

- Each segment is itself divided into pages — combines segmentation's logical/user-view benefit with paging's no-external-fragmentation benefit.
- Similar in spirit to multilevel paging, but segments (unlike the fixed outer-table chunks of multilevel paging) can vary in size according to logical program structure.

Questions

Q1. [Trap] Internal fragmentation is characteristic of which allocation scheme, and external fragmentation of which?

A. Internal→variable partitioning, External→fixed partitioning B. Internal→fixed-size schemes (e.g. paging), External→variable-size contiguous schemes C. Both always occur together D. Neither occurs in modern OSes

Answer: B. **Why:** Fixed-size blocks (paging, fixed partitions) waste space inside an allocated block (internal); variable-size contiguous allocation leaves scattered unusable gaps between blocks (external).

Q2. [Numerical] A system has logical address space of 2^{20} bytes and page size 2^{12} bytes. How many bits are used for the page number?

A. 8 B. 12 C. 20 D. 32

Answer: A. **Why:** Page number bits = $m - d = 20 - 12 = 8$ bits (offset uses the remaining 12 bits).

Q3. [Numerical] TLB access = 10 ns, memory access = 90 ns, hit ratio = 90%. Find the Effective Access Time.

A. 90 ns B. 100 ns C. 109 ns D. 190 ns

Answer: C. **Why:** $EAT = 0.9 \times (10 + 90) + 0.1 \times (10 + 90 + 90) = 0.9 \times 100 + 0.1 \times 190 = 90 + 19 = 109$ ns.

Q4. [Trap] Which statement correctly distinguishes segmentation from paging?

A. Segmentation is physical and fixed-size; paging is logical and variable-size B. Segmentation reflects the user's logical view with variable-size units; paging is a physical, fixed-size, system-level division C. Both are always identical in modern OS design D. Paging suffers external fragmentation; segmentation does not

Answer: B. **Why:** Segmentation exposes the program's natural logical structure (segments of varying size); paging is a physical bookkeeping mechanism using fixed-size frames invisible to the programmer.

Q5. [Numerical] Using the worked segmentation example (P2: limit=212, relocation=880), is a request for offset 212 valid, and why?

A. Valid, physical address = 1092 B. Invalid — offset must be strictly less than the limit C. Valid, physical address = 880 D. Invalid — relocation register is too large

Answer: B. **Why:** The check requires offset < limit; offset=212 equals the limit exactly, which fails the strict inequality, so it traps as illegal.

Q6. Why does multilevel paging use more memory accesses per address translation than single-level paging?

A. It doesn't; multilevel paging is always faster B. Each additional page table level requires its own memory access before reaching the actual data C. Multilevel paging removes the need for a page table entirely D. It only affects TLB size, not access count

Answer: B. **Why:** Address translation must walk each level (outer table → inner table → data), adding one memory access per level compared to single-level paging.

Q7. An Inverted Page Table's size is primarily determined by:

A. The number of logical pages in the largest process B. The number of physical frames in the system C. The number of processes running D. The TLB size

Answer: B. **Why:** It has exactly one entry per physical frame (recording which process/page occupies it), so its size scales with physical memory, not with per-process logical address space.

Q8. [Numerical] Using Weighted/Proportional frame allocation: 3 processes of sizes 20K, 30K, 50K share 40 available frames. How many frames does the 30K process get?

A. 8 B. 10 C. 12 D. 15

Answer: C. **Why:** $a(i) = (s(i)/S) \times m = (30/100) \times 40 = 12$.

Q9. [Trap] A page fault occurs when:

A. A page is accessed and found in the TLB B. A process accesses a page marked invalid (not currently in main memory) C. The CPU cache misses D. The relocation register overflows

Answer: B. **Why:** The valid-invalid bit scheme marks pages not currently resident in memory as invalid; accessing one triggers a page fault, causing the OS to fetch it from disk.

8. Virtual Memory & Page Replacement

8.1 Virtual Memory & Demand Paging

- **Virtual memory** lets a process execute even if it (or the sum of all running processes) is larger than physical memory — only needed pages are resident.
- **Pure Demand Paging:** never bring a page into memory until it is actually referenced (process starts with zero pages loaded, faults its way to a working set).
- Advantages: processes can exceed physical memory size; more processes fit in memory simultaneously (higher multiprogramming); enables easy memory sharing.
- Disadvantages: complex to implement; can cause severe performance loss if mismanaged (**Thrashing**, see §8.5).
- **Valid-invalid bit** (per page-table entry): valid = page is legally in the process's address space AND currently in memory; invalid = either not part of the address space at all, or valid but currently only on disk.

Page Fault Handling Steps

1. Trap to OS (invalid page referenced)
2. Check if reference is valid (illegal → abort) or just not-yet-loaded
3. Find a free frame (or select a victim to evict, if none free)
4. Schedule a disk read to bring the required page into that frame
5. Update the page table (mark valid, set frame number)
6. Restart the instruction that caused the fault

Effective Access Time (Demand Paging)

$$EAT = (1 - p) \times ma + p \times \text{page_fault_service_time} \quad [p = \text{page fault probability, } ma = \text{memory access time}]$$

Worked Example

$ma = 100 \text{ ns}$, page fault service time = $8 \text{ ms} = 8,000,000 \text{ ns}$, $p = 0.001$ (0.1%).

$EAT = (0.999 \times 100) + (0.001 \times 8,000,000) = 99.9 + 8000 = \approx \mathbf{8100 \text{ ns}} (\sim \mathbf{8.1 \mu s})$.

- **TRAP:** Even a TINY page-fault probability (0.1%) dominates EAT because the fault service time is $\sim 100,000\times$ the normal memory access time — this is why minimizing page faults matters so much.

8.2 Page Replacement — Why & The Dirty/Modify Bit

- When no free frame exists, the OS must evict ("replace") a victim page — if it has been modified since loading (**dirty/modify bit set**), it must be written back to disk first (costly); if unmodified, it can simply be discarded (already matches the disk copy).

8.3 Page Replacement Algorithms — Compared on ONE Reference String

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with **3 frames** (hand-traced below; each algorithm operates on the identical string so results are directly comparable):

```
FIFO -> 10 page faults  (evict OLDEST-loaded page)
LRU  -> 9 page faults  (evict LEAST-RECENTLY-used page)
OPT  -> 7 page faults  (evict page used FARTHEST in the future / never again)
```

- **FIFO:** simple (just a queue of load order), but can suffer **Belady's Anomaly** — increasing the number of frames can, for some reference strings, *increase* the fault count (a uniquely FIFO-family pathology).
- **Optimal (OPT):** replace the page not needed for the longest future time — provably the **LOWEST** possible fault rate for a given frame count, but **impossible to implement** (needs future knowledge); used only as a theoretical benchmark for comparison.
- **LRU:** replace the page unused for the longest PAST time — a practical backward-looking approximation of OPT. Implemented via per-page **counters** (timestamp on each reference, evict smallest) or a **stack** (move referenced page to top; LRU page is always at the bottom).
- LRU and OPT both belong to the class of **stack algorithms**: the set of resident pages for n frames is always a subset of the set for $(n+1)$ frames — this property is exactly why stack algorithms **never** suffer Belady's Anomaly (a guarantee FIFO does not have).
- **TRAP:** Belady's Anomaly (more frames $\Rightarrow$ possibly more faults) is a FIFO-specific risk — LRU and OPT are immune to it by construction (stack property).

8.4 Counting-Based Page Replacement

Algorithm	Rule	Weakness
LFU (Least Frequently Used)	Evict the page with the smallest reference count	A page used heavily early then abandoned keeps a high count and wrongly stays resident
MFU (Most Frequently Used)	Evict the page with the HIGHEST count (argues a low-count page was just brought in and will be used soon)	Counter-intuitive; rarely used in practice

- Neither LFU nor MFU is common in real systems — both are expensive to implement and approximate OPT poorly compared to LRU.

8.5 Frame Allocation, Working Set & Thrashing

- Equal allocation:** every process gets $\text{Total_Frames} / N$, regardless of size — simple but ignores actual need.
- Proportional/Weighted allocation:** $a(i) = (s(i) / S) \times m$ where $s(i)$ =process i 's size, S =sum of all sizes, m =total frames (see Ch.7 §7 worked example — same formula).
- Working Set Model:** the set of pages a process actively references within its most recent Δ (window of references) — approximates the process's current "locality." Too small $\Delta \rightarrow$ misses the true locality; too large $\Delta \rightarrow$ overlaps multiple localities and over-allocates.
- Thrashing:** system spends more time servicing page faults than executing useful work. Root cause: too many processes / too little memory per process (multiprogramming pushed too high) $\rightarrow$ each process's allocated frames fall below its working-set size $\rightarrow$ constant faulting $\rightarrow$ CPU utilization drops $\rightarrow$ OS (misreading low CPU utilization as "room for more work") admits MORE processes $\rightarrow$ makes it worse (a vicious cycle).
- TRAP:** The fix for thrashing is to **DECREASE** the degree of multiprogramming (suspend/swap out some processes), NOT increase it — a classic exam trap since low CPU utilization intuitively (but wrongly) suggests "add more processes."

8.6 Local vs Global Replacement

Feature	Local Replacement	Global Replacement
Victim pool	Only the faulting process's OWN frames	ANY frame in the system, even another process's
Isolation	Process performance depends only on its own paging behavior	One process can be affected by another's paging behavior
Frame count per process	Fixed	Can grow/shrink dynamically
Common in practice?	Less common	More common — generally better system-wide throughput

Questions

Q1. [Numerical] For the reference string 7,0,1,2,0,3,0,4,2,3,0,3,2 with 3 frames, how many page faults does FIFO produce?

A. 7 B. 9 C. 10 D. 13

Answer: C. **Why:** Hand-tracing FIFO (evict oldest-loaded on each miss) over all 13 references with 3 frames yields exactly 10 faults (verified step by step).

Q2. [Numerical] For the same reference string and frame count, how many faults does the Optimal algorithm produce?

A. 5 B. 7 C. 9 D. 10

Answer: B. **Why:** OPT always evicts the page needed farthest in the future; tracing this string yields 7 faults — the lowest possible for 3 frames, confirming $\text{OPT} \leq \text{LRU} \leq \text{FIFO}$ here ($7 \leq 9 \leq 10$).

Q3. [Trap] Which page replacement algorithm(s) can exhibit Belady's Anomaly (more frames $\rightarrow$ possibly more faults)?

A. LRU only B. OPT only C. FIFO D. Both LRU and OPT

Answer: C. **Why:** FIFO lacks the "stack property"; LRU and OPT are stack algorithms and are provably immune to Belady's Anomaly.

Q4. [Numerical] $m_a = 200$ ns, page fault service time = 10 ms, page fault rate $p = 0.0005$. Find EAT (approx).

A. 200 ns B. 5200 ns C. 5000 ns D. 10000 ns

Answer: B. **Why:** $\text{EAT} = (1 - 0.0005) \times 200 + 0.0005 \times 10,000,000 \approx 199.9 + 5000 = 5199.9 \approx 5200$ ns.

Q5. [Trap] A system shows falling CPU utilization as more processes are admitted, so the scheduler admits even MORE processes to compensate. This is:

A. The correct fix — always add more processes when CPU utilization drops B. Thrashing — the correct fix is to REDUCE the degree of multiprogramming, not increase it C. A sign the disk is too fast D. Impossible; CPU utilization always rises with more processes

Answer: B. **Why:** This describes classic thrashing — the intuitive "add more work" response worsens it; the real fix is to lower multiprogramming so each process regains enough frames for its working set.

Q6. Which two page-replacement algorithms belong to the "stack algorithm" class and can never suffer Belady's Anomaly?

A. FIFO and LFU B. LRU and OPT C. MFU and FIFO D. LFU and MFU

Answer: B. **Why:** LRU and OPT both satisfy the stack property (frames for n is a subset of frames for $n+1$), which structurally rules out Belady's Anomaly.

Q7. Global page replacement is generally preferred over local replacement in practice because:

A. It fully isolates processes from each other B. It generally yields greater overall system throughput, despite reducing a process's control over

its own fault rate C. It uses less memory D. It eliminates thrashing entirely

Answer: B. **Why:** Global replacement lets frames flow to processes that need them most, usually improving system-wide throughput, at the cost of one process's performance being influenced by others' paging.

Q8. [Trap] The Least Frequently Used (LFU) algorithm can wrongly retain a page in memory when:

A. The page was never referenced at all B. The page was used heavily very early on but has since become unused, keeping a high count that resists eviction C. The frame table is empty D. The reference string is very short

Answer: B. **Why:** LFU relies purely on cumulative counts, so a page with an old burst of heavy use keeps a large count long after it stops being relevant — a known weakness addressed only by extra mechanisms like count-decay.

Q9. In the Working Set model, choosing too LARGE a value of Δ (the window size) causes:

A. The working set to miss part of the true locality B. The working set to overlap multiple distinct localities, reducing its accuracy C. Guaranteed thrashing D. No effect on accuracy

Answer: B. **Why:** A window that's too wide captures pages from more than one locality, making the "working set" a poor approximation of what the process is actually using right now.

9. Disk Scheduling Algorithms

9.1 Why Disk Scheduling Matters

- Seek time (moving the read/write head to the target track/cylinder) dominates total disk access time. Minimizing total head movement across a batch of pending requests improves throughput and response time.
- When the request queue has only one outstanding request, ALL algorithms behave identically (only one place to go) — they only differ meaningfully under load.

9.2 The Algorithms

Algorithm	Rule	Starvation risk
FCFS	Service requests strictly in arrival order	None, but poor seek-time optimization
SSTF (Shortest Seek Time First)	Always jump to the closest pending request	Yes — far-away requests can wait indefinitely (like SJF scheduling)
SCAN ("Elevator")	Sweep to one END of the disk servicing all requests along the way, then reverse	None, but requests just missed wait almost a full sweep
C-SCAN (Circular SCAN)	Sweep to one end servicing requests, then jump back to the START with no servicing on the return, then sweep again	None; more <i>uniform</i> wait times than SCAN
LOOK	Like SCAN but reverses at the last request in that direction, not the physical disk end	None
C-LOOK	Like C-SCAN but jumps back only to the lowest pending request , not the physical disk start	None

- **TRAP:** SSTF is essentially "SJF for disk scheduling" — same greedy-optimal-locally-but-starves-far-requests trade-off.
- **TRAP:** SCAN/C-SCAN go all the way to the **physical end/start of the disk** even if no request is waiting there (wasted movement); LOOK/C-LOOK fix this by only going as far as the last real request.

9.3 Worked Example (classic, verify by hand)

Disk: cylinders 0-199. Head starts at 53, initially moving toward higher numbers.
Pending requests: 98, 183, 37, 122, 14, 124, 65, 67

FCFS (service in given order: 98,183,37,122,14,124,65,67):

$$|98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65|$$

$$= 45+85+146+85+108+110+59+2 = 640$$

SSTF (always nearest next): 53→65→67→37→14→98→122→124→183

$$12+2+30+23+84+24+2+59 = 236 \quad (\text{better than FCFS, but NOT optimal — the theoretical best reordering for this exact set totals 208})$$

TRAP: SSTF is a good greedy heuristic but is **not guaranteed optimal** — it can still leave a shorter overall path on the table (as this example shows: 236 vs the true optimum 208).

SCAN (sweep up to disk end 199, then down to far end 0):

$$(199-53) + (199-14) = 146 + 185 = 331$$

C-SCAN (sweep up to 199, jump to 0 with no service, continue up to 37):

$$(199-53) + 199 + (37-0) = 146 + 199 + 37 = 382$$

LOOK (sweep up only to last request 183, reverse to last request 14):

$$(183-53) + (183-14) = 130 + 169 = 299$$

C-LOOK (sweep up to last request 183, jump directly to lowest request 14, continue up to 37):

$$(183-53) + (183-14) + (37-14) = 130 + 169 + 23 = 322$$

Algorithm	Total head movement (this example)
FCFS	640
SSTF	236
SCAN	331
C-SCAN	382
LOOK	299
C-LOOK	322

9.4 Choosing an Algorithm

- SSTF is common and a natural improvement over FCFS.
- SCAN/C-SCAN suit **heavy-load** systems better (less starvation risk under constant new requests) even though their raw totals may exceed LOOK/C-LOOK in a single static snapshot.
- LOOK/C-LOOK are generally more efficient than their SCAN/C-SCAN counterparts (avoid unnecessary travel to the physical disk ends) but have overhead in tracking the current furthest request.

Questions

Q1. [Numerical] Head at 50 (range 0-199), pending requests {20, 90, 60, 150}, servicing FCFS in that order. Total head movement is:

A. 180 B. 220 C. 260 D. 300

Answer: B. **Why:** $|20-50| + |90-20| + |60-90| + |150-60| = 30+70+30+90 = 220$.

Q2. Which disk scheduling algorithm is described as behaving like the "SJF of disk scheduling," greedily minimizing the NEXT seek but risking starvation of distant requests?

A. FCFS B. SSTF C. SCAN D. C-LOOK

Answer: B. **Why:** SSTF always services the geometrically closest pending request next — the same greedy, starvation-prone logic as Shortest Job First in CPU scheduling.

Q3. [Trap] Which algorithm(s) travel all the way to the physical ends of the disk (0 and max cylinder) even when no request is waiting there?

A. LOOK and C-LOOK only B. SCAN and C-SCAN C. FCFS only D. SSTF only

Answer: B. **Why:** SCAN/C-SCAN always sweep to the physical disk boundary before reversing/jumping; LOOK/C-LOOK instead reverse/jump at the last actual pending request, saving that wasted travel.

Q4. Why does C-SCAN generally provide more *uniform* wait times than plain SCAN?

A. It services fewer requests B. It never reverses direction, always sweeping the same way and jumping back without servicing, so no cylinder waits through two consecutive sweep-passes C. It always starts from cylinder 0 D. It is identical to FCFS

Answer: B. **Why:** Because C-SCAN treats the disk as circular (only ever sweeping in one direction and jumping back without servicing), a request just missed waits roughly one full cycle rather than the uneven wait pattern SCAN produces near the reversal point.

Q5. For the classic example (head=53, requests {98,183,37,122,14,124,65,67}, disk 0-199, initial direction increasing), which algorithm yields the SMALLEST total head movement?

A. FCFS B. SSTF C. SCAN D. LOOK

Answer: B. **Why:** SSTF totals 236 in this example, less than FCFS (640), SCAN (331), and LOOK (299) — though note SSTF is still not the true theoretical optimum (208) for this specific set.

Q6. [Conceptual] Under which condition do FCFS, SSTF, SCAN, and all other disk scheduling algorithms behave identically?

A. When the disk is defragmented B. When there is only one outstanding request in the queue C. When the head starts at cylinder 0 D. Never — they always differ

Answer: B. **Why:** With only one pending request, every algorithm has just one possible next move, so they all degenerate to the same behavior as FCFS in that instant.

10. File Management — Allocation, Free Space, Directories & RAID

10.1 File Allocation Methods

Method	Access supported	Advantage	Disadvantage
Contiguous	Sequential + Direct	Simple, fast, minimal seek between blocks	Severe external fragmentation ; hard to grow a file in place
Linked	Sequential only (efficient)	No external fragmentation; file grows freely as long as free blocks exist	Direct access is slow (must traverse pointer chain from the start); extra space per block for pointers
Indexed	Sequential + Direct	Direct access without external fragmentation (index block gathers all pointers in one place)	Wastes space for small files (index block overhead); needs linked/multilevel index schemes for very large files

- **TRAP:** Linked allocation's single point of failure — a single corrupted/lost pointer breaks the rest of the chain. Partial fixes: doubly linked lists, or storing filename+relative-block-number in each block (both add overhead).
- **Indexed allocation variants** for large files: Linked index blocks (chain of index blocks), Multilevel index (first-level index points to second-level index blocks, which point to data — extendable to more levels), Combined scheme (classic UNIX inode: ~12 direct pointers + single/double/triple indirect pointers — small files need no indirection at all, keeping common-case access fast).

10.2 Free Space Management

Technique	Mechanism	Tradeoff
Bit Vector (Bitmap)	1 bit per block: 1=free, 0=allocated	Simple, efficient at finding n consecutive free blocks, but must be entirely memory-resident to be fast — large disks need a large bitmap
Linked List	Free blocks chained via pointers; only head pointer cached in memory	No bitmap-storage overhead, but traversing the list requires a disk read per block — slow (rarely needed though, since usually only the FIRST free block is needed)

Bit vector size (bytes) = $\text{ceil}(\text{total_blocks} / 8)$
 Example: 1.3 GB disk, 512-byte blocks -> ~2.7M blocks -> bitmap needs > 332 KB
 Example: 1 TB disk, 4 KB blocks -> ~256M blocks -> bitmap needs 256 MB

10.3 File Access Methods

Method	How it works	Best for
Sequential	Read word-by-word from a base pointer that auto-advances	Simple streaming (tape-era design)
Direct / Relative	Jump straight to a block via its block number (fixed-length records)	Random-access needs; faster avg access, but complex to implement, raises security concerns (arbitrary block access)
Indexed Sequential	An index (like a book's index) on top of sequential storage — primary index -> secondary index -> data	Databases: fast direct lookup AND ordered sequential scans together

10.4 Directory Structures

Structure	Key idea	Limitation
Single-Level	All files in one directory	Name collisions once files/users grow
Two-Level	One User File Directory (UFD) per user, indexed by a Master File Directory (MFD)	Solves collisions, but users can't create their own sub-directories
Tree-Structured	Hierarchical folders; absolute or relative path names; 1 bit per entry marks file(0)/subdirectory(1)	Most common today
Acyclic-Graph	Allows sharing via links (shared files/subdirectories) but disallows true cycles	More flexible than a tree, but deletion needs care — reference counting or a link/entry-count so the actual file is deleted only when count = 0
General Graph	Links allowed even where they create cycles	TRAP: cycles can cause infinite loops during traversal — requires cycle detection (checked on every new link, computationally expensive) or garbage collection , or simply restrict links to files only (never subdirectories) to guarantee no cycle can form

- Path name = sequence of directory names from a starting point to the target file. **Absolute** = from root. **Relative** = from current directory.

10.5 File System Mounting & Sharing

- **Mounting:** attach an unmounted file system at a **mount point** (typically an empty directory) so it becomes part of the directory tree.
- **File Sharing — Access Rights:** a graded list from none up to deletion rights (None, Knowledge, Execution, Reading, Appending, Updating, Changing Protection, Deletion).
- **Simultaneous Access:** concurrent writers risk corruption; brute-force fix = whole-file locking when updating — but this raises **mutual exclusion and deadlock** concerns, same as process synchronization.

10.6 File System Implementation

- **On-disk structures:** Boot Control Block, Volume Control Block / **superblock** (volume-wide metadata), per-file **File Control Block (FCB)** / **inode** (owner, size, permissions, block locations).
- **In-memory structures:** mount table, system-wide + per-process open-file tables, directory-structure cache.
- **Layered implementation stack:** I/O Control (device drivers, lowest level) -> Basic File System (issues generic read/write-physical-block commands, manages buffers/cache) -> File Organization Module (maps logical blocks 0..N to physical blocks, tracks free space) -> Logical File System (manages metadata/FCB, everything about a file except its actual content).
- **Directory implementation:** Singly Linked List (simple, but $O(n)$ linear search per operation — slow) vs **Hash Table** (key = hash of filename -> fast average-case $O(1)$ lookup; used by most modern file systems).

10.7 Mass Storage & RAID

- Magnetic disks = bulk secondary storage (rotate ~7000 rpm); SSDs = no moving parts so no seek/rotational latency, faster but pricier per MB, erasable only a limited number of cycles (~100,000); Magnetic tape = large capacity, cheapest per byte, but slow/sequential-only — used for backup/archival, not random access.
- Disk attachment: Host-attached (direct I/O bus, e.g. SCSI/Fibre Channel), Network-Attached Storage/NAS (remote file-level access over TCP/IP, e.g. NFS/CIFS/iSCSI), Storage Area Network/SAN (private high-speed block-level network connecting servers+storage).

RAID 0: striping, NO redundancy -> Read/Write: N (fastest), but ANY 1 disk failure loses all data
RAID 1: mirroring (full duplicate) -> Read: N, Write: slowest disk speed; 50% storage efficiency
RAID 2: bit-level striping + Hamming ECC -> rarely used in practice
RAID 4: block striping + 1 dedicated parity disk -> Read: N-1, Write: bottlenecked at the single parity disk
RAID 5: block striping + parity DISTRIBUTED across all disks -> fixes RAID 4's bottleneck; tolerates 1 disk failure
RAID 6: RAID 5 + a SECOND independent parity block -> tolerates 2 simultaneous disk failures

- **TRAP: RAID 0 has ZERO fault tolerance** despite the "RAID" name — it exists purely for performance/capacity via striping, not redundancy; losing any one disk loses everything.
- RAID only detects/recovers from **disk failure** — it does NOT detect silent data corruption. Filesystems like ZFS add end-to-end checksums (collocated with data pointers) for that, and allocate disks into shared **pools** rather than fixed volumes/partitions.

Questions

Q1. Which file allocation method allows direct access to any block without suffering external fragmentation?

A. Contiguous B. Linked C. Indexed D. None of these

Answer: C. Why: Indexed allocation gathers all block pointers in one index block, allowing direct lookup via the ith entry — any free disk block can satisfy new space requests, avoiding external fragmentation (unlike contiguous allocation).

Q2. [Trap] Why is direct access to the i -th block inefficient under linked allocation?

A. Linked allocation doesn't support direct access syntax B. The system must traverse pointers sequentially from the first block to reach the i -th block, each requiring a disk read C. Linked allocation has no index structure at all, making i undefined D. It is actually just as fast as contiguous allocation

Answer: B. **Why:** Each block only stores a pointer to the next one, so reaching block i means reading i blocks in sequence — there's no way to "jump" directly.

Q3. [Numerical] A disk has 800,000 blocks. How many bytes are needed for a bit-vector free-space map?

A. 8,000 bytes B. 80,000 bytes C. 100,000 bytes D. 800,000 bytes

Answer: C. **Why:** Bitmap size = $\text{ceil}(\text{total_blocks} / 8) = 800,000 / 8 = 100,000$ bytes.

Q4. Which directory structure solves the file-name-collision problem across multiple users but still does not allow users to create their own sub-directories?

A. Single-Level B. Two-Level C. Tree-Structured D. Acyclic-Graph

Answer: B. **Why:** Two-Level directories give each user a private UFD indexed via the MFD, eliminating cross-user name collisions, but each UFD is still flat (no nested sub-directories).

Q5. [Trap] What critical structural difference must a General Graph directory handle that an Acyclic-Graph directory does not?

A. General Graph directories cannot support links at all B. General Graph directories may contain cycles, requiring cycle detection or garbage collection to avoid infinite traversal loops C. General Graph directories are always faster to search D. There is no real difference between them

Answer: B. **Why:** Acyclic-Graph structures allow shared links but are constructed to disallow cycles; General Graph structures permit cycles, which can cause infinite loops during traversal unless actively detected/collected.

Q6. In directory implementation, which approach gives faster average-case file lookup — Singly Linked List or Hash Table — and why?

A. Singly Linked List, because it's simpler B. Hash Table, because it computes a key from the filename for near $O(1)$ average lookup instead of a full linear scan C. Both are equally fast D. Neither supports fast lookup

Answer: B. **Why:** A hash table maps a filename's hash directly to its entry, avoiding the $O(n)$ traversal a linked list requires for every operation.

Q7. [Trap] Which RAID level provides ZERO fault tolerance despite improving performance through striping?

A. RAID 1 B. RAID 0 C. RAID 5 D. RAID 6

Answer: B. **Why:** RAID 0 stripes data across disks with no parity or mirroring — it boosts speed/capacity but the failure of even one disk loses all data; "RAID" here is purely about performance, not redundancy.

Q8. Which RAID level uses block-level striping with parity distributed across ALL disks (rather than a single dedicated parity disk), avoiding a parity-disk write bottleneck?

A. RAID 2 B. RAID 4 C. RAID 5 D. RAID 1

Answer: C. **Why:** RAID 5 distributes parity blocks across all disks in the array, unlike RAID 4's single dedicated parity disk which becomes a write bottleneck; RAID 5 tolerates exactly 1 disk failure.

Q9. How many simultaneous disk failures can RAID 6 tolerate, and why?

A. 0, same as RAID 0 B. 1, same as RAID 5 C. 2, because it maintains two independent parity blocks D. Unlimited, since it uses full mirroring

Answer: C. **Why:** RAID 6 extends RAID 5 with a second, independently computed parity block, allowing reconstruction even if two disks fail simultaneously.

Q10. What is the main disadvantage of Indexed Allocation compared to Linked Allocation?

A. It cannot support direct access B. It suffers from external fragmentation C. It wastes more space on pointer/index overhead, especially for small files D. It cannot be used for large files at all

Answer: C. **Why:** Every file needs its own index block regardless of size, so small files pay a fixed overhead cost that linked allocation avoids (linked allocation only stores one pointer per data block already in use).

Q11. Why does RAID (by itself) fail to protect against silent data corruption, and what is the typical fix?

A. RAID always includes corruption checksums B. RAID only handles whole-disk failures; filesystems like ZFS add end-to-end checksums collocated with data pointers to detect/correct corruption C. RAID corruption protection depends on RAID 0 specifically D. There is no known fix for this problem

Answer: B. **Why:** RAID's redundancy is designed around recovering from a disk failing entirely — it doesn't verify that data itself hasn't silently degraded, which is why modern filesystems add their own checksum layer.

11. Unix OS — Kernel, Commands & Shell Scripting

11.1 Unix Overview & Architecture

- Developed at **AT&T Bell Labs**; key features: **Multituser**, **Multitasking**, **Portable** (written in C), **Hierarchical file system** (root /), permission-based security (**rw**x for owner/group/other), powerful **Shell** for scripting/automation.
- Layered architecture (4 layers, innermost to outermost):

```

Layer 1: Hardware      (CPU, memory, storage, I/O devices)
Layer 2: Kernel        (core – bridges hardware & software; manages memory/process/file/device)
Layer 3: System Programs & Utilities (Shell, compilers, editors, grep/wc/date/who)
Layer 4: Application Layer (user/third-party apps – browsers, DBs, business apps)

```

- Kernel's 3-level internal view: **User Level** (programs request OS services) → **System Call & Library Interface** (bridge; C programs use library functions, Assembly can invoke syscalls directly) → **Kernel Level** (File Subsystem with `open()/read()/write()/close()/chmod()/chown()`, Buffering & Block I/O, Device Drivers, Process Control Subsystem) → **Hardware Level** (generates interrupts that the kernel handles).
- TRAP:** The kernel level is reached via **system calls/traps**, not direct hardware access by user programs — this indirection is what makes Unix secure/stable.

11.2 Unix File System & File Types

- "Everything is a file"** — even hardware devices are represented as special files (usually under `/dev`).
- Every file has a unique **inode** (metadata: size, permissions, disk location) and a unique **pathname**.
- File types and their `ls -l` first-character indicator:

Type	<code>ls -l</code> symbol	Description
Ordinary/Regular	-	User data, programs, text — cannot contain other files
Directory	d	Organizes files/subdirectories; branching point of the tree
Character Special	c	Transfers data 1 byte at a time (keyboard, terminal)
Block Special	b	Transfers data in fixed-size blocks (disks, SSDs)
Pipe (FIFO)	p	One-way data flow connecting two commands
Socket	s	IPC / client-server communication
Symbolic Link	l	Shortcut storing a path to another file; broken if target moves/deleted

- TRAP:** Deleting a **symbolic link** does NOT affect the original file; but if the **original** is deleted/moved, the symlink becomes a **broken link** — the dependency is one-directional.

11.3 Essential Unix Commands (Reference)

```

Navigation:      ls, cd, pwd, mkdir, rmdir
File Manipulation: cp, mv, rm, touch, cat
Text Processing: grep, sed, awk, sort, cut, wc
Text Editors:    vi/vim, nano
Permissions:     chmod, chown, ls -l  (permission string rwxrwxrwx = owner|group|other)
Process Mgmt:    ps, kill, top, nice, bg, fg, jobs
Networking:      ping, netstat, ssh, scp
Sys Admin:       who, df, du, uname

```

- Permission string example: `-rwxr-xr--` → owner=read/write/execute, group=read/execute, other=read-only.

11.4 Shell & Shell Scripting Basics

- Shell** = command-line interpreter; accepts commands, interprets them, requests kernel services, displays results. Also manages scripts, I/O redirection, environment variables.
- A **shell script** = text file of commands executed automatically by the shell; starts with a **shebang** `#!/bin/bash`.
- Variables:** `name="Alice"` (no spaces around `=`); accessed with `$name`; case-sensitive; types = user-defined, environment, read-only, local.
- Special variables:** `$0` =script name, `$1, $2, ...` =positional args, `$#` =argument count, `$*` =all args as one string, `@` =all args as separate values, `$?` =exit status of last command (0=success, non-zero=failure).
- Reading input:** `read varname` (add `-s` for silent/password input).

Operators

```
Arithmetic: + - * / %          (via `expr $a + $b` OR modern $(a + b))
Relational: -eq -ne -gt -lt -ge -le      (numeric comparison)
Logical:    && (AND) || (OR)  ! (NOT)
String:     = (equal) != (not equal) -z (empty) -n (not empty)
File test:  -f (regular file) -d (directory) -r/-w/-x (permissions) -e (exists) -s (not empty)
```

- **TRAP:** In `expr`, spaces around operators are REQUIRED and `*` must be escaped (`\*`) since `*` is a shell wildcard; the modern `$(...)` syntax needs neither.

Control Flow

```
if [ cond ]; then ... fi
if [ cond ]; then ... else ... fi
if [ cond ]; then ... elif [ cond2 ]; then ... else ... fi
case $var in
    pattern1) commands ;;
    *) default ;;
esac

while [ cond ]; do ... done      # runs WHILE condition is true
until [ cond ]; do ... done      # runs UNTIL condition becomes true (opposite of while)
for var in list; do ... done     # fixed number of iterations
break        # exits the loop entirely
continue     # skips to next iteration
```

- **TRAP:** `while` loops **while true**; `until` loops **while false** (i.e., until it becomes true) — easy to mix up.

Functions

```
function_name() {
    commands
    return status_code    # 0-255 ONLY — an exit status, NOT a real return value
}
result=$(function_name)  # capture "returned" data via echo + command substitution
```

- **TRAP:** Shell functions **cannot return actual values** via `return` (unlike C/C++) — `return` only yields an exit status (0–255). To get a computed value back, the function must `echo` it and the caller captures it with `$(...)`.

I/O Redirection

```
>  overwrite output to file
>> append output to file
<  read input from file

|  pipe output of one command as input to another
```

Questions

Q1. In `ls -l` output, a line starts with `d`. This entry is a:

A. Regular file B. Directory C. Symbolic link D. Socket

Answer: B. Why: `d` is the first-character indicator for a directory in `ls -l` output.

Q2. [Trap] If the original file that a symbolic link points to is deleted, what happens to the symbolic link?

A. It automatically deletes itself B. It becomes a broken/invalid link C. It becomes a regular file D. Nothing changes, it still works

Answer: B. Why: A symbolic link only stores a path reference; if the target is gone, the link is broken (dangling), while deleting the LINK itself never affects the original file.

Q3. Which shell loop construct executes its body repeatedly UNTIL a condition becomes true (i.e., while it is false)?

A. for B. while C. until D. case

Answer: C. Why: `until` is the logical opposite of `while` — it keeps looping as long as the condition evaluates false.

Q4. [Trap] A shell function uses `return 25` to try to send back the value 25 to the caller. What actually happens?

A. The caller receives 25 as a normal value B. This fails to compile C. It sets the exit status to 25, retrievable only via `$?` , not as a usable return value D. Shell functions cannot use `return` at all

Answer: C. Why: `return` in shell scripting only sets an exit status (0–255); to pass back computed data, the function must `echo` it and the caller captures it with command substitution `$(...)`.

Q5. Which file-test operator checks whether a file exists AND is a regular file (not a directory)?

A. `-e` B. `-d` C. `-f` D. `-s`

Answer: C. Why: `-f` specifically tests for a regular file; `-e` only checks existence (any type), `-d` tests for directory, `-s` tests non-empty.

Q6. The permission string `-rw-r--r--` on a file means:

A. Owner: read/write, Group: read-only, Other: read-only B. Owner: read-only, Group: read/write, Other: none C. It is a directory D. Everyone can execute it

Answer: A. Why: The 9 characters split into 3 groups of 3 (owner/group/other): `rw-` = read+write, `r--` = read-only, `r--` = read-only; leading `-` confirms it's a regular file.

Q7. What does the pipe operator `|` do in a shell command?

A. Redirects output to a file, overwriting it B. Appends output to a file C. Passes the output of one command as input to the next command D. Reads input from a file

Answer: C. Why: `|` chains commands together, feeding the first command's stdout into the second command's stdin.

12. Linux OS — Kernel, Job Scheduling & Memory Management

12.1 Linux Overview & Architecture

- **Linux** = free/open-source, Unix-inspired kernel (Linus Torvalds). A **distribution (distro)** = Linux kernel + bundled software/tools (Ubuntu, Fedora, Debian, CentOS/RHEL).
- Architecture (bottom to top): **Hardware** → **Kernel** → **System Libraries (glibc etc.)** → **Shell (Bash/Zsh/ksh/csh)** → **System Utilities/Applications**.
- Kernel responsibilities: process/memory/file-system/device management, security & access control, system calls, networking (TCP/IP stack).

Kernel Types (general OS classification, not just Linux)

Type	Design	Trade-off
Monolithic (Linux uses this)	All OS services run in kernel space	High performance (direct calls), but large/complex to maintain
Microkernel	Only essential services in kernel; rest in user space	More secure/modular, but IPC overhead reduces performance
Exokernel	Minimal abstraction; apps get near-direct hardware access	Max flexibility/performance, but much more application complexity
Hybrid	Combines monolithic + microkernel traits	Balances performance, security, modularity — used by many modern OSes

- **TRAP:** Linux's kernel is **Monolithic** (not microkernel) — despite this, it's modular via loadable kernel modules, which sometimes causes confusion.

12.2 Essential Linux Commands (Reference)

```

Directory Listing: ls, ls -l, ls -a
File Management:  cp, mv, rm, touch, mkdir, rmdir
Searching:        find, grep, locate
Permissions:      chmod, chown, ls -l
User Management:  useradd, passwd, usermod, who, whoami
Disk & Storage:   df, du, mount, umount, fdisk
Process Control:  kill, killall, systemctl
Process Monitoring: top, htop, ps aux
Sys Monitoring:  vmstat, dmesg, free -h
Networking:      ping, ifconfig/ip, netstat, ssh, scp
Compression:     tar, gzip, gunzip, zip, unzip

```

12.3 Job Scheduling — cron, at, batch, anacron

- **Job types:** One-time (runs once) vs Recurring (runs repeatedly at fixed intervals).
- **cron:** time-based scheduler for **recurring** jobs. **crond:** the background daemon that continuously checks schedules and actually executes them (cron = the mechanism/table; crond = the running service that reads it).
- **crontab** (cron table) — commands: `crontab -e` (edit), `crontab -l` (list), `crontab -r` (remove all), `crontab -u user -e/-l` (another user's jobs).

```

Crontab format: minute hour day-of-month month day-of-week  command
Example: 0 2 * * * /home/user/backup.sh -> runs daily at 2:00 AM

```

- **Special characters:** `*` = every value, `,` = list of values (e.g., `9,18` = 9 AM and 6 PM), `-` = range (e.g., `9-17` = every hour 9AM-5PM), `/` = step interval (e.g., `*/*5` = every 5 minutes).
- **at / atq / atrm:** for **ONE-TIME** future jobs. `at [time]` schedules it (submit with Ctrl+D); `atq` lists pending at-jobs (shows job IDs); `atrm job_id` cancels one.
- **batch:** one-time job, but runs whenever **system load** permits (load-based, not time-based) — contrast directly with `at` (time-based).
- **anacron:** designed for machines **not always powered on** (e.g., laptops) — if a scheduled job's time was missed (machine was off), anacron runs it once the system is back up; tracks last-execution time. Config file: `/etc/anacrontab`, format `period delay job-identifier command`.
- **TRAP:** **at = one-time, time-based** | **cron = recurring, time-based** | **batch = one-time, load-based** | **anacron = recurring, tolerant of downtime** — all four are commonly confused pairwise; the key differentiators are (one-time vs recurring) × (time-based vs load-based vs downtime-tolerant).

12.4 Linux Memory Management

- Responsibilities: track free/used memory, allocate/deallocate per process, manage virtual memory + paging + swapping, memory protection (one process cannot access another's memory — violation → **Segmentation Fault**).
- Process memory layout** (regions): **Text/Code** (instructions) → **Data** (initialized globals/statics) → **BSS** (uninitialized/zero globals) → **Heap** (dynamic allocation, grows up) → **Memory Mapping** (shared libs) → **Stack** (function calls/locals, grows down).
- Address translation**: CPU generates **Logical/Virtual Address** → **MMU** (Memory Management Unit) → **Physical Address** in RAM.
- Page Table Entry (PTE) fields**: Frame Number, Present/Valid bit, Protection bits (R/W/X), Accessed bit, Dirty bit (modified).
- Single-level page table**: simple, but can become huge for large address spaces. **Multi-level page table**: splits into levels — saves page-table memory (only needed portions allocated) at the cost of more lookups per translation.
- TLB**: caches recent page-table lookups. TLB hit = fast (no page-table walk); TLB miss = slower (must consult page table).
- Demand Paging & Page Fault**: load a page only when referenced; if not present → **Page Fault**. Handling steps: CPU references memory → page not in RAM → hardware raises page fault → OS validates the reference → OS finds/selects a free frame → loads page from disk → updates page table → restarts the interrupted instruction.

Page Replacement Algorithms (shared reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2 — 4 frames)

Algorithm	Rule	Belady's Anomaly?
FIFO	Evict the oldest-loaded page	Yes — more frames can mean MORE faults
Optimal (OPT)	Evict the page used farthest in the future (benchmark only, not implementable)	No
LRU	Evict the Least Recently Used page (based on the past)	No
NRU	Uses Reference(R) + Modified(M) bits; 4 classes (00,01,10,11); evict from lowest-numbered non-empty class	No
Clock/Second-Chance	Circular pointer + reference bit; R=0 → evict; R=1 → give second chance (set R=0), advance pointer	Generally avoids it

- TRAP: Belady's Anomaly** (more frames → more page faults) is a **FIFO-specific** pathology — LRU and Optimal belong to the "stack algorithm" class and never exhibit it.
- Swappiness** (Linux kernel tunable, `/proc/sys/vm/swappiness`, current docs allow range **0–200**, default **60**): higher value = more eager to swap out pages vs reclaiming cache; it's a tuning weight, NOT a direct percentage of RAM to swap.
- Swap Partition** (dedicated, fixed size, slightly faster, needs partition-level setup) vs **Swap File** (regular file inside a filesystem, resizable, easier to manage; need not be contiguous on disk).

12.5 Linux Kernel Memory Allocators

- Buddy Allocator**: manages physical memory in power-of-2-sized page blocks; splits a larger block when a smaller one is needed; merges adjacent same-size free "buddy" blocks on free — reduces external fragmentation for physically-contiguous allocations.
- SLAB Allocator**: caches pre-allocated kernel objects of the same type/size (a cache = multiple "slabs" of objects) — very fast alloc/dealloc by reusing freed objects, but has higher memory overhead (per-CPU caches).
- SLOB Allocator**: simple first-fit free-list allocator for very small/embedded systems — low overhead but slow (linear search) and prone to external fragmentation; **removed from the kernel in Linux 6.8** [VERIFY exact version].
- SLUB Allocator**: simpler, more scalable modern replacement for SLAB — **the default slab allocator in current Linux kernels**; less metadata, better multi-core scalability, supports slab merging.
- TRAP**: Don't assume SLAB is still the default — modern Linux defaults to **SLUB**; SLOB has been removed entirely.

Questions

Q1. Linux's kernel architecture is classified as:

A. Microkernel B. Monolithic C. Exokernel D. Hybrid only

Answer: B. Why: All core OS services (process, memory, file, device management) run in kernel space in Linux — the defining trait of a monolithic kernel (though it supports loadable modules for modularity).

Q2. [Trap] A backup job must run exactly once, at 11 PM tonight only. Which Linux tool is appropriate?

A. cron B. at C. anacron D. crond

Answer: B. Why: `at` schedules one-time jobs for a specific future time; `cron` is for recurring jobs.

Q3. A laptop is frequently powered off overnight and misses its scheduled daily cron job. Which tool is designed to still run the missed job once the system restarts?

A. cron B. at C. anacron D. batch

Answer: C. Why: anacron tracks the last execution time and runs missed periodic jobs once the machine becomes available again — cron simply skips a missed window.

Q4. The crontab entry `* /15 * * * * /script.sh` runs the script:

A. Once every 15 hours B. Once every 15 minutes C. At 3:15 PM daily D. Only on the 15th day of the month

Answer: B. Why: `* /15` in the minute field means "every 15 minutes" (step interval), with all other fields as "every value."

Q5. [Trap] Which page replacement algorithm is specifically known for suffering from Belady's Anomaly (more frames can cause MORE page faults)?

A. LRU B. Optimal C. FIFO D. Clock

Answer: C. **Why:** FIFO uniquely exhibits Belady's Anomaly; LRU, Optimal, and Clock (as LRU/FIFO-improved approximations) do not.

Q6. In the Clock (Second-Chance) page replacement algorithm, when the pointer reaches a page with Reference bit $R=1$, what happens?

A. The page is immediately evicted B. The R bit is cleared to 0, the page is given a second chance, and the pointer moves on C. The page is moved to secondary storage immediately D. The algorithm terminates

Answer: B. **Why:** $R=1$ means recently used — the algorithm spares it, resets R to 0, and continues scanning, rather than evicting a recently-referenced page.

Q7. Which Linux kernel memory allocator is the current default for managing frequently-used kernel objects, having replaced both SLAB and SLOB?

A. Buddy Allocator B. SLAB C. SLOB D. SLUB

Answer: D. **Why:** SLUB is simpler and more scalable than SLAB and is the modern default; SLOB has been removed from the kernel.

Q8. A Linux system has `vm.swappiness` set to a very low value. This means the kernel will:

A. Swap aggressively even under mild memory pressure B. Prefer to avoid swapping, favoring cache reclaim instead C. Disable swap entirely, no exceptions D. Swap exactly that percentage of RAM at all times

Answer: B. **Why:** A lower swappiness value biases the kernel toward reclaiming page cache over swapping out process memory; it's a tuning weight, not a fixed percentage or hard on/off switch.

13. Distributed Systems

13.1 Basics

- **Distributed System:** independent computers, each with its own processor/local memory/**local clock** (no global clock), connected via a network, communicating **only via message passing** (no shared memory) — yet appearing to users as one single coherent system.
- **Distributed File System (DFS):** provides transparent access to files stored across multiple networked computers through one common interface.
- Process **events:** **Internal** (within a process), **Send** (message to another process), **Receive** (message from another process) — the building blocks used to reason about ordering/causality in distributed algorithms.

13.2 Distributed Mutual Exclusion

- Same 3 requirements as single-site critical sections: **Mutual Exclusion** (at most 1 process in CS), **Progress** (a requesting process eventually gets in), **Bounded Waiting/Fairness** (no indefinite wait) — but now must be achieved with NO shared memory, only messages.

Algorithm	Main Idea	Central Coordinator?	Key Feature
Centralized	A coordinator process grants/queues CS requests	Yes	Simple, but single point of failure + bottleneck
Ricart-Agrawala	Fully distributed; broadcast timestamped request to ALL processes, enter only after receiving replies from everyone	No	Request/Reply, timestamp-based
Suzuki-Kasami	A unique token circulates; holding it = permission to enter CS; uses sequence numbers to detect outdated requests	No	Token-based
Token-Based (general)	Token = the sole permission object	No	Must guard against token loss/duplication

- **Message complexity (Suzuki-Kasami):** if the requesting process already holds the token → **0 messages**; if the token must be requested → **~N messages** (broadcast request + token transfer). This "0 or N" framing is the standard exam-level fact.
- Token-based problems: **Token Loss** (processes may wait indefinitely — needs a regeneration mechanism), **Token Duplication** (breaks mutual exclusion if it happens), **Starvation** (possible with poor token-passing policy).
- **TRAP:** Centralized approach has the FEWEST messages per request but the WORST fault-tolerance (single point of failure) — token/Ricart-Agrawala trade more messages for no central dependency.

13.3 Distributed Deadlock

- **Distributed Deadlock:** processes at *different sites* wait indefinitely for resources held by others, forming a cycle that spans multiple machines.
- **Wait-For Graph (WFG):** represents which process is waiting for which other process, extended across sites (a global WFG must be assembled from local views).
- **Phantom Deadlock:** a **FALSE positive** — the system detects a deadlock that no longer actually exists, because network/message delays caused it to observe a **stale, inconsistent snapshot** of global state that *looks* like a cycle even though (e.g.) one of the transactions already released its resource.
- **TRAP:** Phantom deadlock is a problem **unique to distributed** deadlock detection — single-site deadlock detection (Ch.6) works on a single consistent, instantaneous state and never suffers this false-positive issue.

13.4 Agreement Problems

- **Consensus Problem:** all non-faulty processes must agree on a single common value, despite some processes possibly failing (crashing).
- **Byzantine Generals / Byzantine Agreement Problem:** agreement must be reached even when some processes send **arbitrary, contradictory, or malicious** messages (not just silently crashing) — a strictly harder problem than simple crash-fault consensus. [VERIFY] The classical result requires more than $3f$ processes (specifically $\geq 3f+1$) to tolerate f Byzantine-faulty processes.
- **TRAP:** Simple **Consensus** only needs to tolerate crash failures; **Byzantine Agreement** must additionally tolerate processes that actively lie/send conflicting messages — don't treat the two as identical difficulty.

13.5 Distributed Transactions & Two-Phase Commit (2PC)

- **Distributed Transaction:** a transaction that accesses data/resources at **multiple sites**; ACID properties (Ch.12) must still hold across all of them.
- **Two-Phase Commit (2PC):**

Phase 1 (Voting): Coordinator asks every participant "can you commit?"
 Each participant replies YES or NO (and locks its part if YES).
 Phase 2 (Commit/Abort): If ALL replied YES -> coordinator broadcasts COMMIT.
 If ANY replied NO -> coordinator broadcasts ABORT.

- **TRAP:** 2PC is **blocking** — if the coordinator crashes AFTER Phase 1 but before sending the Phase-2 decision, participants who already voted YES **must wait** (they cannot unilaterally decide to commit or abort on their own) — this is 2PC's key weakness, which Three-Phase Commit (3PC) attempts to fix by adding a further non-blocking phase.
- Failure handling in distributed systems generally: **Checkpointing** (periodically save consistent state), **Rollback** (revert to a checkpoint after failure), **Message Logging** (replay messages to reconstruct lost state) — used together for failure recovery. **Replication** (keeping copies of data at multiple sites) improves availability but adds the complexity of keeping replicas consistent during distributed transactions.

Questions

Q1. Which distributed mutual exclusion approach has a single point of failure by design?

A. Ricart-Agrawala B. Suzuki-Kasami C. Centralized D. Token-based (general)

Answer: C. **Why:** The centralized approach relies on one coordinator process; if it fails, no process can be granted access to the critical section.

Q2. [Numerical/Conceptual] In the Suzuki-Kasami algorithm, if the requesting process already holds the token, how many messages are needed to enter the critical section?

A. N B. N-1 C. 0 D. 2N

Answer: C. **Why:** If the process already has the token, no request/reply messages are needed at all — it can enter immediately.

Q3. [Trap] A distributed deadlock detection algorithm reports a cycle in the global Wait-For Graph, but by the time this is detected, the deadlock no longer actually exists. This is called:

A. Livelock B. Phantom Deadlock C. Starvation D. Byzantine Failure

Answer: B. **Why:** Phantom deadlock is a false-positive caused by network delay creating a stale, inconsistent global snapshot — a problem specific to distributed (not single-site) deadlock detection.

Q4. The Byzantine Generals Problem is harder than simple crash-fault Consensus because:

A. It involves more processes always B. Faulty processes may send arbitrary/contradictory/malicious messages, not just crash silently C. It never requires agreement D. It only applies to single-site systems

Answer: B. **Why:** Byzantine faults include active, arbitrary misbehavior (not just stopping), which is strictly harder to tolerate than simple process crashes.

Q5. [Trap] In Two-Phase Commit, the coordinator crashes right after Phase 1 (Voting), where all participants voted YES. What is the participants' situation?

A. Each participant can independently decide to commit B. Each participant can independently decide to abort C. Participants must wait/block since they cannot unilaterally decide D. The transaction is automatically aborted

Answer: C. **Why:** This is 2PC's defining weakness — a coordinator failure after voting leaves "yes" voters blocked, unable to safely commit or abort on their own, since they don't know what other participants decided.

Q6. Which distributed mutual exclusion algorithm is based on broadcasting timestamped requests to all other processes and waiting for all replies before entering the critical section?

A. Centralized B. Ricart-Agrawala C. Suzuki-Kasami D. Token ring only

Answer: B. **Why:** Ricart-Agrawala is the classic fully-distributed, timestamp-based request/reply algorithm requiring replies from every other process.

Q7. Why is a global clock unavailable in a typical distributed system model?

A. Each computer has its own local clock and processor, with no shared memory or synchronized global time source B. Distributed systems don't use clocks at all C. All computers share one master CPU D. Message passing eliminates the need for time

Answer: A. **Why:** By definition, distributed system nodes are independent, each with a local clock; the absence of a global clock is precisely why event ordering (e.g., via logical clocks) becomes a core distributed-systems challenge.